

Machine learning

[Notes by Piai Luca]

Academic Year 2024-2025

Contents

1	Probability review for Discrete Random Variables	1
1.1	Probability refresher	1
1.2	Independent events	2
1.3	Conditional probability	3
1.4	Bayes theorem	3
1.5	Random variables	3
1.5.1	Independence of discrete R.V.	4
1.5.2	Expectation for discrete R.V.	4
1.5.3	Moments of a variable	5
1.5.4	Bernuoulli and Binomial distribution	6
2	Learning Models	9
2.1	Formal model (Statistical Learning)	9
2.2	Loss and Empirical Risk Minimization (ERM)	10
2.2.1	Evaluating the performance of binary classification models	10
2.3	Hypothesis Class and ERM	11
2.4	Probably Approximately Correct (PAC) learning	12
2.4.1	Simplified PAC learning	12
2.4.2	PAC learning	15
2.5	Agnostic PAC Learning	15
2.6	An Optimal Predictor	16
2.7	Beyond binary classification	17
2.7.1	Regression	17
2.7.2	Agnostic PAC learnability for general loss functions	18
3	Linear models	19
3.1	Linear predictors and affine functions	19
3.1.1	Equivalent notation	20

3.2	Linear classification	20
3.2.1	The perceptron algorithm	20
3.2.2	Gradient Descent (GD)	22
3.2.3	Stochastic Gradient Descent (SGD)	24
3.2.4	From SDG to Perceptron	25
3.3	Linear regression	27
3.3.1	Least Squares	27
3.3.2	Linear regression for polynomial regression tasks	31
3.4	Logistic regression	32
3.4.1	Model	32
3.4.2	Loss function	33
4	The Bias-Complexity Tradeoff	35
4.1	No-Free-Lunch Theorem	35
4.1.1	No Free Lunch and Prior Knowledge	36
4.2	Error decomposition	36
4.3	Estimating the L_D	37
5	Model Selection and Validation	39
5.1	Validation	40
5.1.1	Hold Out Set	40
5.1.2	Validation for Model Selection	40
5.1.3	Model-Selection Curve	41
5.1.4	Train-Validation-Test split	43
5.1.5	k-Fold Cross Validation	43
5.1.6	k-Fold Cross Validation for Model Selection	44
5.2	What to do if learning fails?	44
5.2.1	$L_S(h_S)$ is large: underfitting	44
5.2.2	$L_S(h_S)$ is small	45
5.2.3	Main approaches	45
6	Regularization and feature selection	47
6.1	Regularized Loss minimization	47
6.2	ℓ_1 Regularization	47
6.2.1	LASSO = linear regression + ℓ_1 regularization	48
6.3	Tikhonov regularization	48
6.3.1	Ridge Regression = linear regression + Tikhonov regularization	49

6.4	Feature selection	49
6.4.1	Subset Selection algorithm	50
6.4.2	Forward Selection algorithm	51
6.4.3	Backward Selection algorithm	51
6.5	Subset, Forward, Backward Selection with Validation Data	52
6.6	Subset, Forward, Backward Selection with cross-validation	54
7	Support Vector Machines	55
7.1	Introduction	55
7.1.1	Classification and margin	55
7.1.2	Linearly separable training set	56
7.1.3	Separating Hyperplane and Margin	56
7.1.4	Margin and Support Vectors	56
7.2	Support Vector Machine: Hard-SVM	57
7.2.1	The homogeneous case and Support Vectors	58
7.3	Support Vector Machine: Soft-SVM	58
7.4	Implementing Soft-SVM using SGD	61
7.5	Duality (Hard-SVM)	61
7.6	Kernel Trick for SVM	62
7.6.1	Commonly used Kernels	63
7.6.2	Choice of Kernel	64
7.7	SVM for Regression	64
8	Neural Networks	67
8.1	Neuron and activation function	67
8.2	Neural Networks: feedforward neural networks	67
8.3	Notation	68
8.3.1	Point of View of one node	68
8.3.2	Neural network: formalism	69
8.4	Hypothesis Set of a Neural Network	69
8.5	Expressive power of neural network	70
8.6	Runtime of Learning Neural Networks	72
8.7	SGD for Neural Networks	73
8.7.1	Notation	73
8.7.2	Forward Propagation Algorithm	74
8.7.3	Learning NN parameters	74
8.7.4	Compute sensitivities vectors	76

8.7.5	Backpropagation Algorithm	77
8.8	Regularized neural networks: squared weight decay regularizer	77
8.9	Geometric Intuition	78
9	Deep Learning	79
9.1	Convolutional Neural Networks (CNNs)	79
10	Learning via Uniform Convergence	83
10.1	Uniform Convergence and Learnability	83
10.2	Finite Classes are Agnostic PAC Learnable	85
11	VC-Dimension	89
11.1	Restrictions	89
11.2	VC-dimension and Shattering	90
11.3	The Fundamental Theorems of Statistical Learning	90
12	Clustering	93
12.1	Unsupervised Learning	93
12.2	Clustering	93
12.2.1	Model for clustering	94
12.3	Cost Minimization Clustering	94
12.3.1	k-Means Clustering	94
12.3.2	k-Medoids objective and k-Median objective	95
12.4	Algorithm for k-Means clustering	95
12.4.1	Lloyd's Algorithm	96
12.4.2	Algorithm k-Means++	96
12.5	Choice of number k of clusters	97
12.5.1	Silhouette	98

Chapter 1

Probability review for Discrete Random Variables

1.1 Probability refresher

Definition 1 (Probability space). *A probability space has three components:*

1. *A sample space Z , which is the set of all possible outcomes of the random process modeled by the probability space (e.g. $Z = \{H, T\}$, referred to coins flipping);*
2. *A family F of sets (e.g. $F = \{\emptyset, \{H\}, \{T\}, \{H, T\}\}$, where $\{H, T\}$ means H or T) representing the allowable events, where each event A is a subset of $Z : A \subseteq Z$ (F must be a σ -algebra...)*
3. *A probability distribution $D : F \rightarrow [0, 1]$ that satisfies the following conditions:*
 - (a) $D[Z] = 1$;
 - (b) *Let $A_1, A_2, A_3, \dots$ be any finite or countably infinite sequence of pairwise mutually disjoint events (so they can't occur together) ($A_i \cap A_j = \emptyset$ for all $i \neq j$) :*

$$D \left[\bigcup_{i \geq 1} A_i \right] = \sum_{i \geq 1} D[A_i]$$

Definition 2 (σ -algebra). *F is a σ -algebra in the space of Z if F is a family of sets of elements $\in Z$ such that:*

1. $Z \in F$;
2. *if $A \in F \Rightarrow A^c \in F$, where $A^c = \text{not } A$;*

3. if $A = \bigcup_i A_i$ with $A_i \in F \forall i$, then $A \in F$.

Examples of σ -algebra:

$$F_1 = \{\emptyset, Z\}$$

$$F_2 = \{\emptyset, A, A^c, Z\} \text{ with } A \subset Z \text{ "}\delta\text{-algebra generated by the event } A\text{"}$$

$$F_3 = \mathbb{P}(Z) \text{ power set of } Z, \text{ i.e. all possible subsets of } Z$$

Example of probability space (Z, F, D) :

$$\text{sample space } Z = \{H, T\}$$

$$\text{events } F = \{\emptyset, H, T, Z\}$$

$$\text{probabilities } P(\emptyset) = 0 \quad P(\{H\}) = P(\{T\}) = 0.5 \quad P(\{H, T\}) = 1$$

Notation:

- We write $E \cup F$ for the occurrence of either E or F .
- We write $E \cap F$ for the occurrence of both E and F .

$$Pr[E \cup F] = Pr[E] + Pr[F] - P[E \cap F] \quad (1.1)$$

Theorem 1 (Boole's law or Union Bound inequality). *For any finite or countable set of events $E_1, \dots, E_k$ the probability that at least one of the events happens is no greater than the sum of the probabilities of the individual events.*

$$Pr \left[\bigcup_i E_i \right] \leq \sum_i Pr[E_i]$$

1.2 Independent events

Definition 3 (Independent events). *Two events E and F are independent ($E \perp F$) if and only if*

$$Pr[E \cap F] = Pr[E] \cdot Pr[F].$$

More generally, events $E_1, \dots, E_k$ are mutually independent if and only if for any subset $I \subseteq [1, k]$

$$Pr \left[\bigcap_{i \in I} E_i \right] = \prod_{i \in I} Pr[E_i]$$

1.3 Conditional probability

Definition 4 (Conditional probability). *The conditional probability that event E_1 occurs given that event E_2 occurs is*

$$Pr(E_1|E_2) = \frac{Pr(E_1 \cap E_2)}{Pr(E_2)}$$

The conditional probability is only well-defined if $Pr(E_2) > 0$.

Theorem 2 (Law of Total Probability). *Let $E_1, \dots, E_n$ be mutually disjoint events in the sample space Z , and $\bigcup_{i=1}^n E_i = Z$ i.e. E_i are a partition of Z . Then:*

$$Pr(B) = \sum_{i=1}^n Pr(B \cap E_i) = \sum_{i=1}^n P(B|E_i)P(E_i).$$

1.4 Bayes theorem

Theorem 3 (Bayes theorem). *The Bayes theorem gives a mathematical rule for inverting conditional probabilities.*

$$Pr[E_1|E_2] = \frac{P[E_2|E_1]P[E_1]}{P[E_2]}$$

More in general, assume that $E_1, \dots, E_n$ are mutually disjoint events in the sample space Z , and $\bigcup_{i=1}^n E_i = Z$ i.e. E_i are a partition of Z . Then

$$P(E_j|B) = \frac{P(E_j \cap B)}{Pr(B)} = \frac{Pr(B|E_j)P(E_j)}{Pr(B)} = \frac{Pr(B|E_j)P(E_j)}{\sum_{i=1}^n P(B|E_i)P(E_i)}.$$

1.5 Random variables

Definition 5 (Random variable). *A (scalar) random variable $X(z)$ on a sample space Z is a real-valued function on Z ; that is, $X : z \in Z \rightarrow \mathbb{R}$.*

A random variable is function whose domain is the sample space Z of a random experiment and whose codomain is the set of real numbers. Informally, a random variable is a numerical observation resulting from the outcome of a random experiment.

We distinguish the random variables in two groups:

- **Continuous random variable:** codomain is continuous.
 - Cumulative Distribution Function (CDF) or Distribution: $F_X(x) = Pr[X \leq x]$
 - Probability Density Function (PDF): $p_X = \frac{\partial F_X(x)}{\partial x}$ thus $F_X = \int_{-\infty}^x p_X(a) da$

- **Discrete random variable:** codomain is finite or countable.
 - Cumulative Distribution Function (CDF) or Distribution: $F_X(x) = Pr[X \leq x] = \sum_{k \leq x} p_X(k)$
 - Probability Mass Function (PMF): $p_X(x) = Pr[X = x]$

Probability density function gives the probability that a continuous random variable will lie between a certain specified interval. Probability mass function denotes the probability that a discrete random variable will take on a particular value.

1.5.1 Independence of discrete R.V.

Definition 6. Two discrete random variables X and Y are independent ($X \perp Y$) if and only if

$$Pr((X = x) \cap (Y = y)) = Pr(X = x) \cdot Pr(Y = y)$$

for all values x and y . Similarly, discrete random variables $X_1, \dots, X_k$ are mutually independent if and only if for any subset $I \subseteq [1, k]$ and any values $x_i, i \in I$

$$Pr(\mathbf{x}) = \prod_{i \in I} P(X_i = x_i) = \prod_{i \in I} p_{X_i}(x_i).$$

1.5.2 Expectation for discrete R.V.

Definition 7 (Expected value). The expectation of a discrete random variable X is

$$E[X] = \sum_x x \cdot p_X(x)$$

Theorem 4. Given two random variables X_1 and X_2

$$E[X_1 + X_2] = E[X_1] + E[X_2].$$

Corollary 1. Given a constant $c \in \mathbb{R}$ and a random variable X

$$E[c \cdot X] = c \cdot E[X].$$

Theorem 5 (Expectation of a function). Let $g(X)$ be a function of a discrete random variable X . Then

$$E[g(X)] = \sum_x g(x) \cdot p_X(x)$$

1.5.3 Moments of a variable

For a random variable X we define:

- **Mean:** $m_X = E[X]$
- **Variance:** $\sigma_X^2 = E[(X - m_X)^2] = E[X^2] - m_X^2 = \text{Var}[X]$
- **k-th moment:** $E[X^k]$

For a vector valued r.v. $\mathbf{X} \in \mathbb{R}^n$ we define:

- Expectation:

$$E[\mathbf{X}] = \begin{bmatrix} m_{X_1} \\ \vdots \\ m_{X_n} \end{bmatrix}$$

- Instead of the variance we have the *covariance matrix*:

$$\Sigma = \mathbb{E}[(\mathbf{X} - m_{\mathbf{X}})(\mathbf{X} - m_{\mathbf{X}})^T] = \begin{bmatrix} \sigma_{X_1}^2 & \sigma_{X_1, X_2} & \cdots & \sigma_{X_1, X_n} \\ \sigma_{X_2, X_1} & \sigma_{X_2}^2 & \vdots & \sigma_{X_2, X_n} \\ \vdots & \vdots & \vdots & \vdots \\ \sigma_{X_n, X_1} & \sigma_{X_n, X_2} & \cdots & \sigma_{X_n}^2 \end{bmatrix}$$

where σ_{X_i, X_j} is the **covariance** of X_i and X_j

$$\sigma_{X_i, X_j} = \text{Cov}(X_i, X_j) = E[(X_i - m_{X_i})(X_j - m_{X_j})]$$

Theorem 6. If X_1, X_2 are independent $\Rightarrow \sigma_{X_1, X_2} = 0$.

The opposite isn't necessary true.

Theorem 7 (Properties of Mean, Variance, etc.). *Properties:*

- $E[X_1 + X_2] = E[X_1] + E[X_2]$
- $\text{Var}[aX + b] = a^2 \text{Var}[X]$
- $\text{Var}[X_1 + X_2] = \text{Var}[X_1] + \text{Var}[X_2] + 2\sigma_{X_1, X_2}$

Corollary 2. If $\sigma_{X_1, X_2} = 0$ (so X_1, X_2 are independent) $\Rightarrow \text{Var}[X_1 + X_2] = \text{Var}[X_1] + \text{Var}[X_2]$.

1.5.4 Bernuoulli and Binomial distribution

Consider an event A with probability p . $X_1, \dots, X_n$ are independent and identically distributed (iid) random variables that are indicator functions of event A :

$$X_i(z) = \begin{cases} 1 & \text{if } z \in A \\ -1 & \text{if } z \notin A \end{cases}$$

for example A could be "coin flip is head". Each X_i is a **Bernuoulli random variable** of parameter p :

$$X_i \sim B(p).$$

There fore the probability of the event A is p and

- $E[X_i] = p$
- $Var[X_i] = p(1 - p)$

Then

$$S_n = \sum_{i=1}^n X_i$$

is a **Binomial random variable** of parameters n, p :

$$S_n \sim Bin(n, p)$$

The probability that S_n equals to some value k can be seen as the number of ways to chose k entities at random from a set of n entities (in every possible order):

$$Pr[S_n = k] = \binom{n}{k} p^k (1 - p)^{n-k}.$$

also:

- $E[S_n] = np$
- $Var[S_n] = np(1 - p)$

Theorem 8 (Markov's inequality). *Let X be a non negative random variable. Then, $\forall a > 0$:*

$$Pr(X \geq a) \leq \frac{E[X]}{a}$$

From which we can derive:

Theorem 9 (Chebyshev's inequality). *Let X be a random variable with $E[X] = \mu$ and $\text{Var}[X] = \sigma^2$. Then:*

$$\Pr[|X - \mu| > \varepsilon] \leq \frac{\sigma^2}{\varepsilon^2}$$

We define the **relative frequency** as follows:

$$f_n(A) = \frac{S_n}{n} = \frac{1}{n} \sum_{i=1}^n X_i$$

$$E[f_n(A)] = p, \quad \text{Var}[f_n(A)] = \frac{p(1-p)}{n}.$$

Using the Chebyshev's inequality we have:

$$\Pr[|f_n(A) - p| > \varepsilon] \leq \frac{p(1-p)}{n\varepsilon^2}$$

and

$$\lim_{n \rightarrow +\infty} f_n(A) = p.$$

Theorem 10 (Law of Large Numbers). *Let $X_i, i = 1, \dots, n$ be i.i.d. with $\mathbb{E}[X_i] = \mu$ and $\text{Var}[X] = \sigma^2 < +\infty$. Then*

$$\lim_{n \rightarrow +\infty} \Pr \left[\left| \frac{1}{n} \sum X_i - \mu \right| > \varepsilon \right] = 0$$

Remarks:

- Remark 1:

$$\lim_{n \rightarrow +\infty} f_n(A) = \Pr[A]$$

- Remark 2:

$$\begin{aligned} \Pr[A | B] &= \frac{\Pr[A \cap B]}{\Pr[B]} = \lim_{n \rightarrow +\infty} \frac{f_n(A \cap B)}{f_n(B)} \\ &= \frac{f_n(A \cap B)}{f_n(B)} = \frac{S_n(A \cap B)}{S_n(B)} \end{aligned}$$

it's the fraction of times $A \cap B$ happens among those in which B happens.

Chapter 2

Learning Models

2.1 Formal model (Statistical Learning)

A learner (us, or the machine) has access to:

1. Domain set X : set of all possible objects to make predictions about
 - $x \in X$ is called instance, usually represented by a vector of features
 - X is the instance space
2. Label set Y : set of possible labels (e.g $\{-1, +1\}$ or $\{0, 1\}$).
3. Training data $S = ((x_1, y_1), \dots, (x_m, y_m))$: finite sequence of labeled domain points. S is called *training set* and represents the learner's input.
4. Learner's output h : prediction rule $h : X \rightarrow Y$. The function h (sometimes indicated as $\hat{f}$) is called *predictor*, *hypothesis*, or *classifier*. We define $A(S)$ as the *prediction rule* produced by learning algorithm A when training set S is given to it.
5. Data-generation model: instances are generated by some probability distribution D and labeled according to a function. We define:
 - D is the distribution over X that is NOT known to the learner. (The population distribution).
 - $f : X \rightarrow Y$ is the "real" labeling function that is not known to the learner.
 - y_i is the label of instance $x_i : y_i = f(x_i)$, for all $i = 1, \dots, m$.
 - Each point in training set S is composed by a sample x_i according to D and his label $y_i = f(x_i)$.

6. Measures of success: error of a classifier, thus the probability it does not predict the correct label on a random data point generate by distribution D .

2.2 Loss and Empirical Risk Minimization (ERM)

Given domain subset $A \subset X$, we define $D(A)$ as the probability of observing a point $x \in A$.

Definition 8 (Loss). *The error of a prediction rule $h : X \rightarrow Y$ is:*

$$L_{D,f}(h) \stackrel{\text{def}}{=} \mathbb{P}_{x \sim D}[h(x) \neq f(x)] \stackrel{\text{def}}{=} D(\{x : h(x) \neq f(x)\}).$$

It's the probability of observing a point $x \in D$ such that the label predicted by the model $h(x)$ is different from the true label $f(x)$.

$L_{D,f}(h)$ (often f is omitted, i.e. $L_D(h)$) is called: generalization error, true error, risk or loss. This value indicate the ability of the learner to generalize on the entire population, thus the ability of finding the correct label for every possible sample $x \in D$. The **goal** is to find a function $h : X \rightarrow Y$ that minimizes the generalization error $L_{D,f}(h)$, but the problem is that D and f are unknown, so how we compute the value? We consider the error on the training data set S and therefore the output is h_S : the function that minimize the error on training data.

Definition 9 (Training error). *The training error can be defined as follows:*

$$L_S(h) = \frac{|\{i : h(x_i) \neq y_i, i = 1, \dots, m\}|}{m}$$

where m is the number of instances in S .

The training error is also called *empirical error* or **empirical risk** and there are many ways to define this parameter, it depends on the application; in the definition above we have define it as the average of the mistakes.

2.2.1 Evaluating the performance of binary classification models

Definition 10 (Accuracy). *The accuracy is defined as follows:*

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Using the definition 9 of the training error $L_S(h)$, we have that $\text{Accuracy} = 1 - L_D(h)$.

Definition 11 (Matthews's correlation coefficients). *The MCC is defined as follows:*

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC is a balanced measure useful for problems in which classes have different size. Often the loss function is defined as $1 - MCC$.

Other statistics we can observe:

Definition 12 (Sensitivity or recall). *The sensitivity is defined as follows:*

$$Sensitivity = \frac{TP}{TP + FN}$$

Definition 13 (Specificity). *The specificity is defined as follows:*

$$Specificity = \frac{TN}{TP + FN}$$

Definition 14 (Precision or Positive Predictive Value). *The PPV is defined as follows:*

$$PPV = \frac{TP}{TP + FP}$$

Definition 15 (Negative Predictive Value). *The NPV is defined as follows:*

$$NPV = \frac{TN}{TN + FN}$$

None of the performance scores above is useful if used as a loss function, and that's because none of them consider all the possible cases (like MCC does).

2.3 Hypothesis Class and ERM

The goal of finding an hypothesis h that minimizes the empirical error, can be written as follows:

$$ERM_H \in \arg \min_{h \in H} L_S(h) \quad (2.1)$$

where H is a *restricted set* of hypothesis, also known as *hypothesis class*. Each $h \in H$ is a function $h : X \rightarrow Y$.

When we're searching for an hypothesis, the problem that we want to avoid is the **overfitting**. We say that an hypothesis suffer of overfitting when it performs very well on

the training set but very poorly on instances never seen before, thus $L_S(h_S)$ is a small value but in the other hand $L_{D,f}(h_S)$ isn't so small. So the question now is: which hypothesis classes H do not lead to overfitting?

2.4 Probably Approximately Correct (PAC) learning

2.4.1 Simplified PAC learning

Lets assume H is a *finite class*, so $|H| < +\infty$ and let h_S be the output of $ERM_H(S)$ ($h_S \in ERM_H(S)$, see 2.1). The finite hypothesis class is a type of class that is never used in the real world problems. For example lets define $H = \{h_{a,b}(\mathbf{x}) : h_{a,b}(\mathbf{x}) = \text{sign}(ax_1, bx_2)\}$. This is a linear hypothesis class and as we can see $|H| = +\infty$ even if the hypotheses functions are very simple.

Also consider the following assumptions:

- **Realizability:** there exists $h^* \in H$ such that $L_{D,f}(h^*) = 0$. Observe that the realizability implies also that $L_S(h^*) = 0$.
- **i.i.d:** the m examples in the training set are independently and identically distributed according to D . We use $S \sim D^m$ to say that the training set S contains m examples generated by the distribution D .

We'll use them to prove the following theorem. So our question now is: can we *learn* (i.e. find using ERM) the hypothesis h^* ?

The PAC does not guarantee that we can find h^* , instead it guarantees that the hypothesis h_S is approximately correct with a certain probability. This is because the samples in S come from D , so we can't be 100% sure that the hypothesis found h_S has $L_D(h_S) = 0$. For instance: lets say that we have two hypothesis h_1, h_2 and a very large data set. Lets suppose that we know that $L_{D,f}(h_1) = 0$ and $L_{D,f}(h_2) \approx 0$, thus h_2 makes some mistakes in predicting the labels for some input x_i . But if $x_i \notin S$ than it could be that $L_S(h_1) = L_S(h_2) = 0$ and we can't distinguish the two hypotheses.

Theorem 11 (Simplified PAC). *Let H be a finite hypothesis class. Let $\delta, \epsilon \in]0, 1[$ and let the size of the training set $|S| = m \in \mathbb{N}$ such that*

$$m \geq \frac{\log(|H|/\delta)}{\epsilon}.$$

Then for any f and any D for which the realizability assumption holds, with probability $\geq 1 - \delta$ we have that for every ERM hypothesis h_S it holds that

$$L_{D,f}(h_S) \leq \varepsilon.$$

The parameters defined in the theorem above are:

- *Accuracy parameter ε* : the smaller it is and the more accurate the hypothesis. It indicates the upper bound to the true error.
- *Confidence parameter δ* : $1 - \delta$ is the probability that the hypothesis $h_S \in \text{ERM}_H(S)$, obtained by minimizing the empirical risk, has a true error $L_{D,f}(h_S)$ bounded above by ε .

Note that the theorem above its valid for any true function f and any distribution D , and this is good for our purpose because we don't know anything about them. In other words, the theorem says that with a finite H we can "almost always" find a "good hypothesis" if we have "enough data".

Proof. Let $S|_X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$ be the instance of the training set S (so we don't consider the labels). We want to find an upper bound to

$$D^m(\{S|_X : L_{D,f}(h_S) > \varepsilon\}).$$

Let

$$H_B = \{h \in H : L_{D,f}(h) > \varepsilon\}$$

(that is, the set with *bad hypothesis*) and

$$M = \{S|_x : \exists h \in H_B, L_S(h) = 0\}$$

(that is, the set with *misleadings samples*).

Since we have the realizability assumption:

$$L_S(h_S) = 0 \Rightarrow L_{D,f}(h_S) > 0 \text{ only if some } h \in H_B \text{ has } L_S(h) = 0$$

If that's the case, the output h_S is a "bad" hypothesis and our training data $S|_x$ must be in the set M :

$$\{S|_X : L_{D,f}(h_S) > \varepsilon\} \subseteq M \quad (*).$$

Note that:

$$M = \bigcup_{h \in H_B} \{S|_X : L_S(h) = 0\}.$$

Therefore (from $\star$):

$$\begin{aligned} D^m(\{S|_X : L_{D,f}(h_S) > \varepsilon\}) &\leq D^m(M) = D^m\left(\bigcup_{h \in H_B} \{S|_X : L_S(h) = 0\}\right) \\ &\leq \sum_{h \in H_B} D^m(\{S|_X : L_S(h) = 0\}) \quad (\star\star) \end{aligned}$$

using the union bound property. Now we can consider a single element of the sum above by fixing $h \in H_B$. We know that

$$L_S(h) = 0 \Leftrightarrow \forall i = 1, \dots, m : h(\mathbf{x}_i) = f(\mathbf{x}_i)$$

therefore

$$\begin{aligned} D^m(\{S|_X : L_S(h) = 0\}) &= D^m(\{S|_X : \forall i = 1, \dots, m : h(\mathbf{x}_i) = f(\mathbf{x}_i)\}) \\ &= \prod_{i=1}^m D(\{\mathbf{x}_i : h(\mathbf{x}_i) = f(\mathbf{x}_i)\}) \quad (\text{because } \mathbf{x}_i \text{ are iid from } D) \quad (\star\star\star) \end{aligned}$$

Consider some i , $1 \leq i \leq m$:

$$\begin{aligned} D(\{\mathbf{x}_i : h(\mathbf{x}_i) = f(\mathbf{x}_i)\}) &= 1 - D(\{\mathbf{x}_i : h(\mathbf{x}_i) \neq f(\mathbf{x}_i)\}) \\ &= 1 - L_{D,f}(h) \\ &\leq 1 - \varepsilon \quad (\text{since } h \in H_B) \\ &\leq e^{-\varepsilon} \quad (\text{Taylor expansion of } e^x) \end{aligned}$$

Combining this with $(\star\star\star)$:

$$D^m(\{S|_X : L_S(h) = 0\}) \leq \prod_{i=1}^m e^{-\varepsilon} = e^{-m\varepsilon}.$$

Combining this with $(\star\star)$:

$$\begin{aligned} D^m(\{S|_X : L_{D,f}(h_S) > \varepsilon\}) &\leq D^m(M) \leq \sum_{h \in H_B} e^{-m\varepsilon} = |H_B| \cdot e^{-m\varepsilon} \leq |H| \cdot e^{-m\varepsilon} \\ &\leq |H| e^{-\log(\frac{|H|}{\delta})} = |H| \cdot \frac{\delta}{|H|} = \delta. \end{aligned}$$

Given the choice $m \geq \frac{1}{\varepsilon} \log(\frac{|H|}{\delta})$. So we have demonstrate that the probability of having a training set with generalization error grater than ε , using the function h_S obtained by ERM, is lower or equal to δ . $\square$

2.4.2 PAC learning

Definition 16 (PAC learnability). *A hypothesis class H is PAC learnable if there exist a function $m_H : (0, 1)^2 \rightarrow \mathbb{N}$ and a learning algorithm such that for every $\delta, \epsilon \in]0, 1[$, for every distribution D over X , and for every labeling function $f : X \rightarrow \{0, 1\}$, if the realizability assumption holds with respect to H, D, f , then when running the learning algorithm on $m \geq m_H(\epsilon, \delta)$ i.i.d. generate by D and labeled by f , the algorithm returns a hypothesis h such that, with probability $\geq 1 - \delta$ (over the choice of examples): $L_{D,f}(h) \leq \epsilon$*

Corollary 3. *Every finite hypothesis class H is PAC learnable with sample complexity*

$$m_H(\epsilon, \delta) \leq \left\lceil \frac{\log(|H|/\delta)}{\epsilon} \right\rceil.$$

2.5 Agnostic PAC Learning

The Agnostic PAC Learning is a more general learning model: the **realizability assumption is removed**. Informally, the realizability assumption says that the label is fully determined by the instance x . In many real application this assumption is too strong, so instead we can consider the following *relaxation*: D is a probability distribution over $X \times Y$, that is, D is the joint distribution over domain points and labels. With this relaxation we don't have to consider any "real function" f .

So we can define:

- D_x : the marginal distribution over domain points.
- $D((x, y)|x)$: conditional distribution over labels for each domain point¹.

Given x , label y is obtained according to a condition probability $P[y|x]$. Because of the fact that there isn't a true function f , we have to change the definition of true error and the definition of empirical error.

Definition 17. *With D that is a probability distribution over $X \times Y$, the true error (or risk) is:*

$$L_D(h) \stackrel{\text{def}}{=} P_{(x,y) \sim D}[h(x) \neq y]$$

Remember that D is not known to the learner, he knows only about the training set S .

¹The same sample x can produce different labels y .

Definition 18. We define the empirical risk as:

$$L_S(h) \stackrel{\text{def}}{=} \frac{|\{i, 0 \leq i \leq m : h(x_i) \neq y_i\}|}{m}$$

The definition of empirical risk is the same as before but the meaning is different: $L_S(h)$ is the probability that for a pair (x_i, y_i) taken uniformly at random from S the event " $h(x_i) \neq y_i$ " holds. From this observation we can derive an interesting property:

$$E_S[L_S(h)] = L_D(h) \quad \text{with a very large } m = |S|. \quad (2.2)$$

Definition 19 (Agnostic PAC Learnability). A hypothesis class H is agnostic PAC learnable if there exist a function $m_H : (0, 1)^2 \rightarrow \mathbb{N}$ and a learning algorithm such that for every $\delta, \epsilon \in (0, 1)$, for every distribution D over $X \times Y$, when running the learning algorithm on $m \geq m_H(\epsilon, \delta)$ iid examples generated by D the algorithm returns a hypothesis h such that, with probability $\geq 1 - \delta$ (over the choice of the m training examples):

$$L_D(h) \leq \min_{h' \in H} L_D(h') + \epsilon$$

In other words, removing the realizability assumption, leads us to find a predictor that is near the best one: the generalization loss $L_D(h)$ is close to the best possible value² provided by some hypothesis h' in the class H that we're considering. So the definition above is a generalization of the previous learning model.

2.6 An Optimal Predictor

Given a probability distribution D over $X \times \{0, 1\}$, the best predictor is the **Bayes Optimal Predictor**:

$$f_D(\mathbf{x}) = \begin{cases} 1 & P[y = 1 | \mathbf{x}] \geq 1/2 \\ 0 & \text{otherwise.} \end{cases}$$

Proposition 1. For any classifier $g : X \rightarrow \{0, 1\}$, it holds $L_D(f_D) \leq L_D(g)$.

The problem is that we cannot use such predictor because we don't know D , so we don't know $P[y = 1 | \mathbf{x}]$, $\forall \mathbf{x}$.

²In PAC learning, because of the realizability assumption, we had: $\min_{h' \in H} L_D(h') = 0$

2.7 Beyond binary classification

So far we've only considered *binary classification*, where $y \in Y = \{0, 1\}$. Other type of learning problems that we will consider are:

- **Multiclass classification:** classification problems with more than 2 labels.
- **Regression:** the labels are real numbers: $y \in Y = \mathbb{R}$.

For multiclass problems we can use the same properties that we have introduced in the previous sections.

2.7.1 Regression

In regression problems we have that $x \in X = \mathbb{R}^p$ as usual, and, as we have mentioned previously, $y \in Y = \mathbb{R}$. Also (as before): the training set $S = ((x_1, y_1), \dots, (x_m, y_m))$; the learner's output is $h : X \rightarrow Y$. For regression problems we can't use the same definition of loss function that we use in classification problems, this is because it lacks the concept of "near value". For example:

$$h(\mathbf{x}_i) = 3.69, \quad h(\mathbf{x}_i) = 3690, \quad y_i = 3.7$$

we need to define a function that tells us that 3.69 is better than 3690 because it's closer to 3.7.

Definition 20 (Loss function). *Given any hypotheses set H and some domain Z , a loss function is any function $l : H \times Z \rightarrow \mathbb{R}^+$.*

In general, we define:

- **Risk function:** the expected loss of a hypothesis $h \in H$ with respect to D over Z .

$$L_D(h) \stackrel{\text{def}}{=} E_{z \sim D}[l(h, z)] \quad (2.3)$$

(In our case $(\mathbf{x}, y) = z \in Z = X \times Y$).

- **Empirical risk:** expected loss over a given samples set $S = (z_1, \dots, z_m) \in Z^m$.

$$L_S(h) \stackrel{\text{def}}{=} \frac{1}{m} \sum_{i=1}^m l(h, z_i) \quad (2.4)$$

Let us define some common loss functions used in practice:

- **0-1 loss:** $Z = X \times Y$

$$l_{0-1}(h, (x, y)) \stackrel{\text{def}}{=} \begin{cases} 0 & h(x) = y \\ 1 & h(x) \neq y \end{cases} \quad (2.5)$$

It's commonly used in binary or multiclass classification.

- **Squared loss:** $Z = X \times Y$

$$l_{sq}(h, (x, y)) \stackrel{\text{def}}{=} (h(x) - y)^2 \quad (2.6)$$

It's commonly used in regression.

In real world problem we choose the loss function based on the application that we want to implement. This is a crucial point: different loss function leads us to different hypothesis.

In regression problems, instead of the squared loss, we could use the absolute value. In real world ML application this function is never used: we can't compute the derivative of the absolute value, so we can't find the best hypothesis.

2.7.2 Agnostic PAC learnability for general loss functions

Definition 21. A hypothesis class H is agnostic PAC learnable with respect to a set Z and a loss function $l : H \times Z \rightarrow \mathbb{R}^+$ if there exists a function $m_H : (0, 1)^2 \rightarrow \mathbb{N}$ and a learning algorithm such that for every $\delta, \epsilon \in (0, 1)$, for every distribution D over Z , when running the learning algorithm on $m \geq m_H(\epsilon, \delta)$ iid examples generated over D , the algorithm returns a hypothesis h such that, with probability $\geq 1 - \delta$ (over the choice of the m training examples):

$$L_D(h) \leq \min_{h' \in H} L_D(h') + \epsilon$$

where $L_D(h) = E_{z \sim D}[l(h, z)]$.

This is the most general definition of PAC learnability. In the previous sections, we focused exclusively on the loss function $l_{0,1}$. However, as we can see, agnostic PAC learnability can be defined for any general loss function l .

Chapter 3

Linear models

3.1 Linear predictors and affine functions

In this chapter we will consider $X = \mathbb{R}^d$, also we define a **linear (affine) functions** as follows:

$$L_d = \left\{ h_{\mathbf{w},b} : \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R} \right\}, \quad |L_d| = +\infty \quad (3.1)$$

where

$$h_{\mathbf{w},b}(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = \left(\sum_{i=1}^d w_i x_i \right) + b. \quad (3.2)$$

Each element of L_d is a function $\mathbf{x} \rightarrow \langle \mathbf{w}, \mathbf{x} \rangle + b$; b is called **bias**.

It will be convenient also to use the notation

$$L_d = \{ \mathbf{x} \rightarrow \langle \mathbf{w}, \mathbf{x} \rangle + b, \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R} \}$$

which reads as follows: L_d is a set of functions, where each function is parameterized by $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$, and each such function takes as input a vector $\mathbf{x}$ and returns as output the scalar $\langle \mathbf{w}, \mathbf{x} \rangle + b$.

The different hypothesis classes of linear predictors are compositions of a function $\phi : \mathbb{R} \rightarrow Y$ on L_d . For example, in binary classification, we can choose ϕ to be the *sign* function ($\phi(z) = \text{sign}(z)$), and for regression problems, where $Y = \mathbb{R}$, ϕ is simply the identity function ($\phi(z) = z$).

3.1.1 Equivalent notation

Given $\mathbf{x} \in X, \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$, define:

$$\begin{aligned}\mathbf{w}' &= (b, w_1, w_2, \dots, w_d) \in \mathbb{R}^{d+1}, & (\text{adding bias as first element}) \\ \mathbf{x}' &= (1, x_1, x_2, \dots, x_d) \in \mathbb{R}^{d+1} & (\text{adding a one as first element})\end{aligned}$$

Then we can define:

$$h_{\mathbf{w},b}(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = \langle \mathbf{w}', \mathbf{x}' \rangle$$

So we will consider bias term as part of $\mathbf{w}$ and assume $\mathbf{x} = (1, x_1, x_2, \dots, x_d)$ when needed, with $h_{\mathbf{w}}(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle$. In the following section we will use this notation.

3.2 Linear classification

Consider (binary) classification: $X = \mathbb{R}^d, Y = \{-1, 1\}$; we use the 0 – 1 loss as function loss. The hypothesis class in linear classification are halfspaces defined as follows:

$$HS_d = \text{sign} \circ L_d = \{x \rightarrow \text{sign}(h_{\mathbf{w},b}(\mathbf{x})) : h_{\mathbf{w},b} \in L_d\}.$$

In other words, each halfspace hypothesis in HS_d is parameterized by $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ and upon receiving a vector $\mathbf{x}$ the hypothesis returns the label $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle)$.

Our goal is to find a good hypothesis by minimizing the training error (ERM). The first algorithm that we introduce is called **perceptron algorithm**. Before introducing the algorithm we make the assumption that our data are **linearly separable**, this means that there exists $\mathbf{w}$ such that $y_i \langle \mathbf{w}, \mathbf{x}_i \rangle > 0 \forall i = 1, \dots, m$ (realizability assumption for the training set)¹.

3.2.1 The perceptron algorithm

The perceptron algorithm takes as input the training set and returns in output the model $\mathbf{w}$ (see algorithm 0).

Note that:

$$\begin{aligned}y_i \langle \mathbf{w}^{(t+1)}, \mathbf{x}_i \rangle &= y_i \langle \mathbf{w}^{(t)} + y_i \mathbf{x}_i, \mathbf{x}_i \rangle \\ &= \underbrace{y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle}_{\leq 0} + \underbrace{\|\mathbf{x}_i\|^2}_{> 0}\end{aligned}$$

so the update "guides" $\mathbf{w}$ to be "more correct" on the instance $(\mathbf{x}_i, y_i)$. Also note that after the update, instances that were previously classified correctly may now be misclassified. The

¹If $y_i \langle \mathbf{w}, \mathbf{x}_i \rangle > 0$ for all $i = 1, \dots, m$ where m is the size of our training set $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$, implies that all points are classified correctly by the model $\mathbf{w}$.

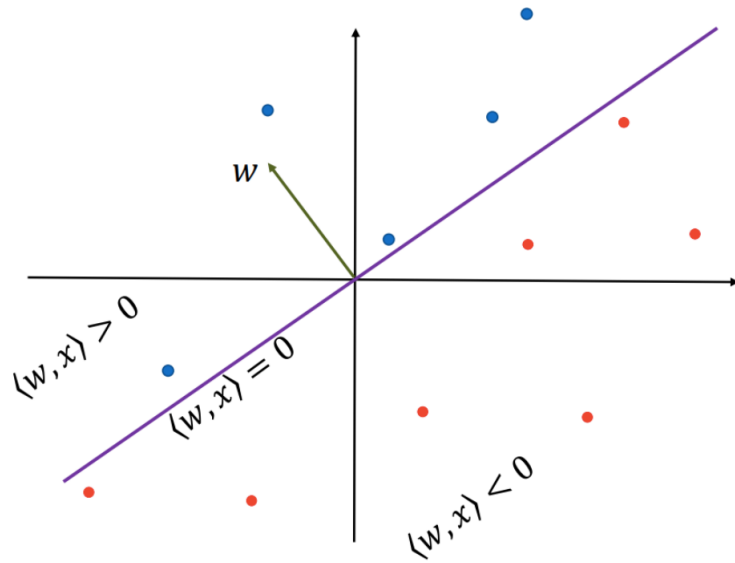


Figure 3.1: Example with $d = 2$. Each hypothesis forms a hyperplane that is perpendicular to the vector $\mathbf{w}$ and intersects the vertical axis at the point $(0, -b/w_2)$. The instances that are "above" the hyperplane (the blue dots) are labeled positively; the instances that are "below" the hyperplane (the red dots), are labeled negatively.

Algorithm 1 Perceptron Algorithm

Input: A training set $(x_1, y_1), \dots, (x_m, y_m)$
Initialize: $\mathbf{w}^{(1)} = (0, \dots, 0)$
for $t = 1, 2, \dots$ **do**
 if $\exists i$ such that $y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle \leq 0$ **then** $\triangleright$ If $\mathbf{x}_i$ is misclassified by the current $\mathbf{w}^{(t)}$
 $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + y_i \mathbf{x}_i$ $\triangleright$ Update
else
 return $\mathbf{w}^{(t)}$ $\triangleright$ All the instances in the training set are correctly classified
end if
end for

algorithm converges to a solution as long as the realizability assumption holds.

Proposition 2. Assume that $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$ is linearly separable, let:

- $B = \min\{\|\mathbf{w}\| : y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \geq 1 \ \forall i, i = 1, \dots, m, \}$, and
- $R = \max_i \|\mathbf{x}_i\|$.

Then the Perceptron algorithm stops after at most $(RB)^2$ iterations (and when it stops it holds that $\forall i, i \in \{1, \dots, m\} : y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle > 0$, all the instances are correctly classified).

In other words, we can prove that the perceptron terminates if the data are linearly separable. If the data are not separable, then we can run the algorithm for some time and keep the best

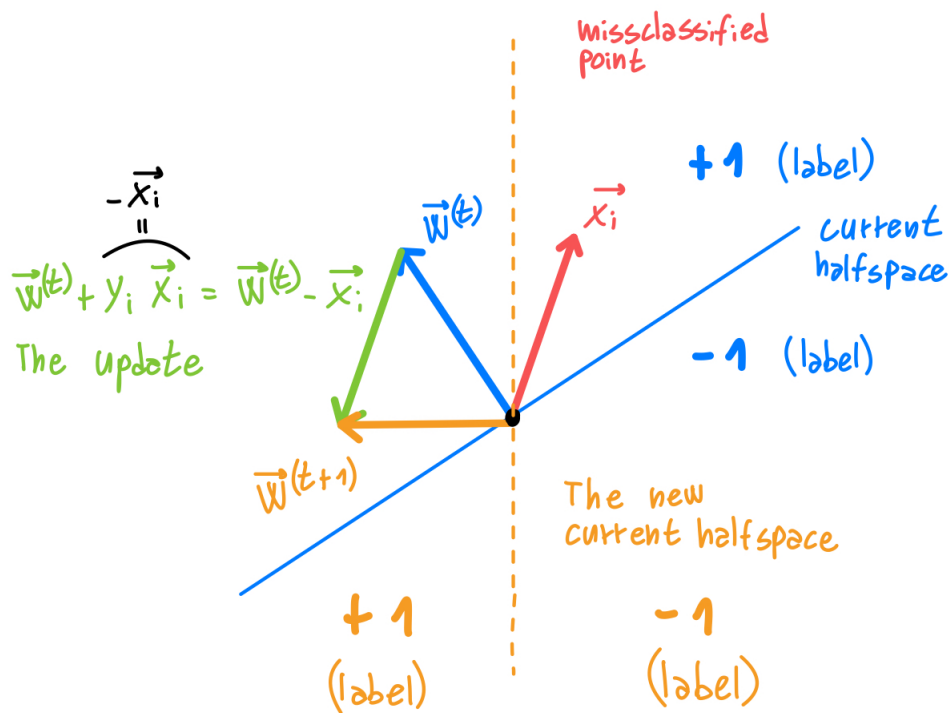


Figure 3.2: Geometrical interpretation of the update: after the update the instance $\mathbf{x}_i$ is correctly classified ($y_i \langle \mathbf{w}, \mathbf{x}_i \rangle > 0$).

solution found up to that point.

As we can see the perceptron algorithm is very easy to implement, but there are some problem. If the data are linearly separable the algorithm, termination is guaranteed, but in the other hand it may require a number of iteration that is exponential in d . In this case, other approaches (e.g. Integer Linear Programming) might be better to find ERM solution. Another problem is that, if the data are separable, there could exist multiple solutions and the one picked by the algorithm depends on the starting value.

3.2.2 Gradient Descent (GD)

Assume you want to solve a binary classification problem with linear models using the loss $l(\underbrace{\mathbf{w}}_{=h_{\mathbf{w},b}}, (\mathbf{x}, y)) = \max\{0, -y \langle \mathbf{w}, \mathbf{x} \rangle\}$. As usual our goal is to find the model/hypothesis with the smallest training error. To achieve our goal we can use the gradient descent algorithm.

The gradient descent represent a general approach for minimizing a differentiable convex

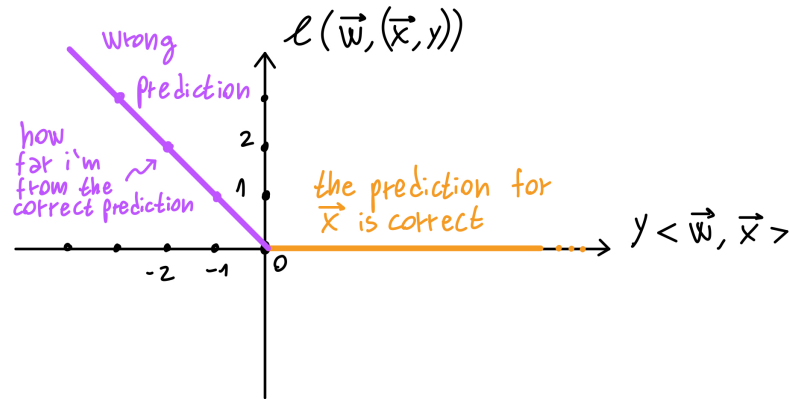


Figure 3.3: If the predicted label is correct, then the loss is 0; if the predicted label is wrong, then the loss is proportional to how far we are from the correct prediction.

function $f(\mathbf{w})$.

Definition 22 (Gradient). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a differentiable function. The gradient $\nabla f(\mathbf{w})$ of f at $\mathbf{w} = (w_1, \dots, w_d)$ is

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f(\mathbf{w})}{\partial w_1}, \dots, \frac{\partial f(\mathbf{w})}{\partial w_d} \right).$$

Intuitively the gradient points in the direction of the greatest rate of increase of f around $\mathbf{w}$.

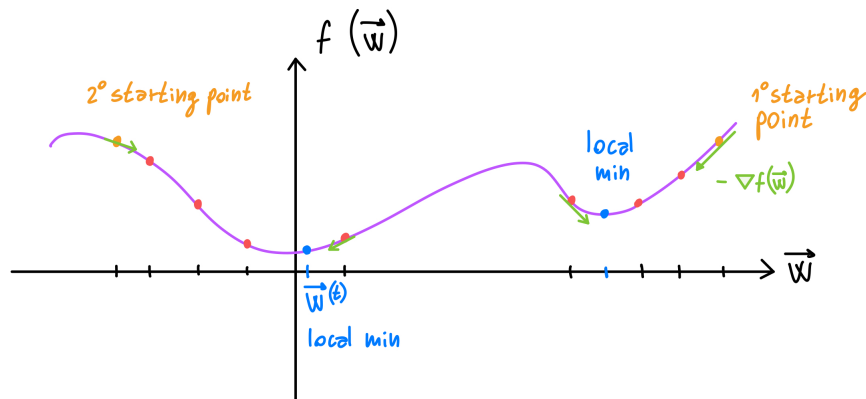


Figure 3.4

In our case the function $f(\mathbf{w})$ is the training error:

$$L_S(h_{\mathbf{w}}) = \frac{1}{m} \sum_{i=1}^m l(\mathbf{w}, (x_i, y_i)) \quad (3.3)$$

that is, L_S is the average of loss computed for each point in the training set S (note that in the equation above, the variable is $\mathbf{w}$, instead $\mathbf{x}$ and y are fixed).

Algorithm 2 Gradient Descent Algorithm

Parameter: $\eta \in \mathbb{R}, \eta > 0$
 $\mathbf{w}^{(0)} \leftarrow \mathbf{0}$
for $t \leftarrow 0$ to $T - 1$ **do**
 $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla f(\mathbf{w}^{(t)})$ ▷ Update the vector
end for
return $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^T \mathbf{w}^{(t)}$

As we can see the algorithm returns the average of all the vector $\mathbf{w}$ computed. Other alternatives are: $\mathbf{w}^{(T)}$ (the last one) or $\arg \min_{\mathbf{w}^{(t)} \in \{\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(T)}\}} f(\mathbf{w}^{(t)})$ (the best one, i.e. the one that minimizes the f function). Returning $\bar{\mathbf{w}}$ is useful for nondifferentiable functions. The parameter η is called **learning rate**. Sometimes depends on time $\eta^{(t)}$ (e.g. "move" more at the beginning than at the end). Under some assumptions on f , the gradient descent algorithm always return a *good* value of $\bar{\mathbf{w}}$.

3.2.3 Stochastic Gradient Descent (SGD)

As we saw in the previous subsection, we can use the GD algorithm with $f(\mathbf{w}) = L_S(h_{\mathbf{w}})$ to find a $\mathbf{w}$ that minizes the training error. The problem is that $\nabla f(\mathbf{w})$ depends on all pairs $(\mathbf{x}_i, y_i)$ $i = 1, \dots, m$ and if m is big, then the computation of the gradient may require a long time.

Algorithm 3 Stochastic Gradient Descent Algorithm

Parameter: $\eta \in \mathbb{R}, \eta > 0$
 $\mathbf{w}^{(0)} \leftarrow \mathbf{0}$ ▷ Or choose random values
for $t \leftarrow 0$ to $T - 1$ **do**
 choose $\mathbf{v}_t$ at random from distribution such that $E[\mathbf{v}_t | \mathbf{w}^{(t)}] \in \nabla f(\mathbf{w}^{(t)})$; ▷ $\mathbf{v}_t$ has *expected value* equal to the gradient of $f(\mathbf{w}^{(t)})$
 $\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} - \eta \mathbf{v}_t$; ▷ Update the vector
end for
return $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^T \mathbf{w}^{(t)}$

In stochastic gradient descent we do not require the update direction to be based exactly on the gradient. Instead, we allow the direction to be a random vector and only require that its expected value at each iteration equals the gradient direction.

We need to choose $\mathbf{v}_t$ such that $E[\mathbf{v}_t | \mathbf{w}^{(t)}] \in \nabla f(\mathbf{w}^{(t)})$. How we do that? We can simply pick a single random $(\mathbf{x}_i, y_i) \in S$ and then we compute $\mathbf{v}_t \in \nabla l(\mathbf{w}^{(t)}, (\mathbf{x}_i, y_i))$ (the $\in$ operator can be seen as an $=$). Due to the fact that we compute the gradient for only one point $(\mathbf{x}_i, y_i)$, the time required is much less than the one required by GD.

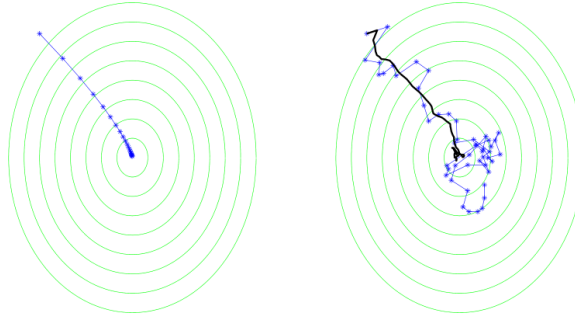


Figure 3.5: An illustration of the gradient descent algorithm (left) and the stochastic gradient descent algorithm (right). For the stochastic case, the black line depicts the averaged value of $\mathbf{w}$ (i.e. $\bar{\mathbf{w}}$).

3.2.4 From SDG to Perceptron

In the beginning of section 3.2 we have considered a binary classification problem with linear models and we have defined the loss function $l(\mathbf{w}, (\mathbf{x}, y)) = \max\{0, -y\langle \mathbf{w}, \mathbf{x} \rangle\}$. Now, how we can find the ERM solution? We use SGD, let's see how we can use it.

In order to use the SDG, we have to take a value i uniformly at random from $\{1, \dots, m\}$. Let $(\mathbf{x}', y') = (\mathbf{x}_i, y_i)$ be the corresponding point in the training set; consider the vector $\nabla l(\mathbf{w}, (\mathbf{x}', y'))$. We can show that $E[\nabla l(\mathbf{w}, (\mathbf{x}', y'))] = \nabla L_S(\mathbf{w})$:

Proof.

$$\nabla L_S(\mathbf{w}) = \nabla \left(\frac{1}{m} \sum_{i=1}^m l(\mathbf{w}, (\mathbf{x}_i, y_i)) \right) = \frac{1}{m} \sum_{i=1}^m \nabla l(\mathbf{w}, (\mathbf{x}_i, y_i))$$

For SDG we have:

$$\begin{aligned} E[\nabla l(\mathbf{w}, (\mathbf{x}', y'))] &= \sum_{i=1}^m \overbrace{Pr[(\mathbf{x}', y') = (\mathbf{x}_i, y_i)]}^{1/m \text{ (uniform distribution)}} \cdot \nabla l(\mathbf{w}, (\mathbf{x}_i, y_i)) \\ &= \frac{1}{m} \sum_{i=1}^m \nabla l(\mathbf{w}, (\mathbf{x}_i, y_i)) \\ &= \nabla L_S(\mathbf{w}) \end{aligned}$$

□

Now, looking at the pseudocode of the SDG algorithm, we want to compute the value $\mathbf{v}_t = \nabla l(\mathbf{w}^t, (\mathbf{x}_i, y_i))$. So how do we actually compute the gradient of the loss function in the point

$(\mathbf{x}_i, y_i)$? Lets consider a general $\mathbf{w}$ (without the t parameter). We have:

$$l(\mathbf{w}, (\mathbf{x}, y)) = \max\{0, -y\langle \mathbf{w}, \mathbf{x} \rangle\}$$

$$\nabla l(\mathbf{w}^t, (\mathbf{x}_i, y_i)) = \begin{cases} 0 & y_i \langle \mathbf{w}, \mathbf{x}_i \rangle > 0 \\ \nabla(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle) & \text{otherwise.} \end{cases}$$

So in the first case $\mathbf{x}_i$ is correctly classified by the model $\mathbf{w}$ so there is no update (i.e. $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)}$); in the second case, $\mathbf{x}_i$ is misclassified by the current model $\mathbf{w}$, so the algorithm update the model (i.e. $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla l(\mathbf{w}, (\mathbf{x}_i, y_i))$).

Lets assume $\mathbf{x}_i$ is misclassified, so we need to compute the value of the gradient:

$$\nabla(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle) \stackrel{\text{def}}{=} \left[\frac{\partial(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle)}{\partial w_1}, \dots, \frac{\partial(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle)}{\partial w_d} \right]^T.$$

We have

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ \vdots \\ x_{id} \end{bmatrix}, \quad -y_i \langle \mathbf{w}, \mathbf{x}_i \rangle = -y_i \sum_{j=1}^d w_j x_{ij}$$

so the function is actually a sum of monomials of degree 1, therefore the gradient is very easy to compute:

$$\begin{aligned} \frac{\partial(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle)}{\partial w_j} &= -y_i \cdot x_{ij} \\ \Rightarrow \nabla(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle) &= [-y_i \cdot x_{i1}, \dots, -y_i \cdot x_{id}]. \end{aligned}$$

We can rewrite the pseudocode of the SDG and modify the line that update the current model:

Algorithm 4 Stochastic Gradient Descent Algorithm (Perceptron)

Parameter: $\eta \in \mathbb{R}, \eta > 0$

$\mathbf{w}^{(0)} \leftarrow \mathbf{0}$

▷ Or choose random values

for $t \leftarrow 0$ to $T - 1$ **do**

pick i uniformly at random from $\{1, \dots, m\}$

if $y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle < 0$ **then**

 ▷ i.e. $\mathbf{x}_i$ is misclassified by $\mathbf{w}^{(t)}$

$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)} + \eta y_i \mathbf{x}_i;$

 ▷ Update the vector

else

$\mathbf{w}^{(t+1)} \leftarrow \mathbf{w}^{(t)}$

end if

end for

return $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=0}^{T-1} \mathbf{w}^{(t)}$

As we can see, the SDG (Perceptron) algorithm is very similar to the Perceptron algorithm.

Perceptron	SDG perceptron
Choose a point that is misclassified	Choose a point at random
η is fixed to value 1	η is a parameter
Return the "best" $\mathbf{w}^{(t)}$	Return $\bar{\mathbf{w}}$ (the average)

Table 3.1: Differences between Perceptron and SDG perceptron.

Actually, the second and third rows do not represent real differences; the primary difference lies in the first row: the SDG perceptron selects a point $(\mathbf{x}_i, y_i)$ at random, regardless of whether it is misclassified. However, selecting a correctly classified point is inefficient, as it results in no update to the model. Consequently, random selection can be a waste of time. To accelerate the SDG perceptron, we can instead choose a misclassified point uniformly at random in each iteration. With this adjustment, the SDG perceptron effectively becomes the perceptron itself.

3.3 Linear regression

In regression learning problems, we have that $\mathbf{x} \in X = \mathbb{R}^d$ and the labels $y \in Y = \mathbb{R}$. In *linear regression* we consider an hypothesis class that contains linear function:

$$H_{reg} = L_d = \{\mathbf{x} \mapsto \langle \mathbf{w}, \mathbf{x} \rangle + b, \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}.$$

As we said in the previous chapter, a common choice for the loss function is the *squared-loss*:

$$l(h, (\mathbf{x}, y)) \stackrel{def}{=} (h(\mathbf{x}) - y)^2.$$

which tells us how far we are from predicting the correct label for a certain $\mathbf{x}$. Therefore the empirical risk function is the following (i.e. the training error):

$$L_S(h) = \frac{1}{m} \sum_{i=1}^m (h(\mathbf{x}_i) - y_i)^2$$

which we will call **Mean Squared Error (MSE)**.

3.3.1 Least Squares

How we can find a ERM hypothesis? We can use the **Least Squares algorithm**. Least squares is the algorithm that solves the ERM problem for the hypothesis class of linear regression predictors with respect to the squared loss. The ERM problem with respect to this class, given

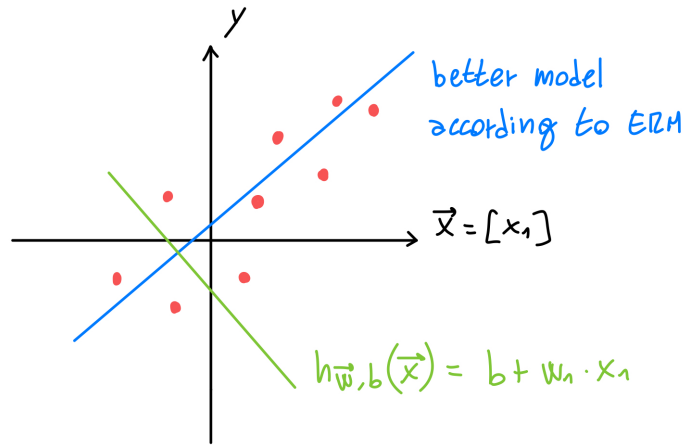


Figure 3.6

a training set $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$, is to find

$$\arg \min_{\mathbf{w}} L_S(h_{\mathbf{w}}) = \arg \min_{\mathbf{w}} \frac{1}{m} \sum_{i=1}^m (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)^2.$$

An equivalent formulation (obtained by removing the $\frac{1}{m}$) is

$$\arg \min_{\mathbf{w}} L_S(h_{\mathbf{w}}) = \arg \min_{\mathbf{w}} \sum_{i=1}^m (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)^2. \quad (3.4)$$

In other words, we're looking for the $\mathbf{w}$ that minimizes the *Residual Sum of Squares* (RSS).

RSS in matrix form The Residual Sum of Squares can be rewritten in a matrix form. Let $\mathbf{X}$ be a $m \times (d+1)$, where the rows² are the vectors $\mathbf{x}_i \in S|_x \forall i = 1, \dots, m$:

$$\mathbf{X} = \begin{bmatrix} \dots & \mathbf{x}_1 & \dots \\ \dots & \mathbf{x}_2 & \dots \\ \dots & \vdots & \dots \\ \dots & \mathbf{x}_m & \dots \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$$

²The matrix $\mathbf{X}$ has $d+1$ columns because of the bias b .

X is called **design matrix**. Also let define the vector of the observed value $\mathbf{y}$ as follows:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}.$$

Now we can define the RSS using the matrix notation:

$$\sum_{i=1}^m (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)^2 = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

And now the problem of finding the $\mathbf{w}$ that minimizes the RSS is described as follows:

$$\arg \min_{\mathbf{w}} \text{RSS}(\mathbf{w}) = \arg \min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) \quad (3.5)$$

Computation of the gradient of the RSS The Figure 3.7 shows the $\text{RSS}(\mathbf{w})$ when $d = 1$.

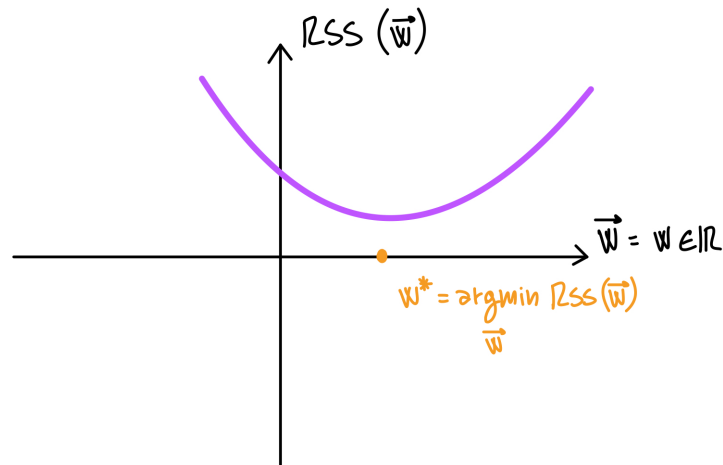


Figure 3.7: The function $\text{RSS}(\mathbf{w})$ in one dimension ($d = 1$ and $\mathbf{w} = w \in \mathbb{R}$) is just a parabola: a convex function $\Rightarrow$ the local minimum coincides with the global minimum.)

In order to minimize the $\text{RSS}(\mathbf{w})$ (our objective function) we have to compute the gradient $\frac{\partial \text{RSS}(\mathbf{w})}{\partial \mathbf{w}}$ with respect to $\mathbf{w}$ and compare it to 0.

$$\frac{\partial \text{RSS}(\mathbf{w})}{\partial \mathbf{w}} = -2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

The equation $-2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) = 0$ is equivalent to:

$$\begin{aligned} -2\mathbf{X}^T(\mathbf{y} - \mathbf{X}\mathbf{w}) = 0 &\Rightarrow \mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y} \\ \text{if } \mathbf{X}^T\mathbf{X} \text{ is invertible} &\Rightarrow \mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \end{aligned}$$

The algorithm that find the model $\mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$ that minimizes the objective function $RSS(\mathbf{w})$ is the following:

1. Compute $\mathbf{X}^T\mathbf{X}$: is the product of $(d+1) \times m$ matrix and $m \times (d+1)$ matrix. As result we obtain a $(d+1) \times (d+1)$ matrix.
2. Compute $(\mathbf{X}^T\mathbf{X})^{-1}$.
3. Compute $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$: is the product of $(d+1) \times (d+1)$ matrix and $(d+1) \times m$ matrix. As result we obtain a $(d+1) \times m$ matrix.
4. Compute $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$: is the product of $(d+1) \times m$ matrix and $m \times 1$ matrix. As result we obtain a $(d+1) \times 1$ matrix.

The most expensive operation is the inversion (cubic cost); however the matrix $\mathbf{X}^T\mathbf{X}$ has $(d+1)$ rows and columns so that isn't a really big number ($m \gg (d+1)$).

The generalized inverse of A The problem is: if $\mathbf{X}^T\mathbf{X}$ is not invertible, how we find $\mathbf{w}$ such that $\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$? We have to use the **generalized inverse of the matrix**.

Let $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ and $\mathbf{A}^+$ be the *generalized inverse* of $\mathbf{A}$, i.e.: $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$.

Proposition 3. If $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is no invertible, then $\hat{\mathbf{w}} = \mathbf{A}^+\mathbf{X}^T\mathbf{y}$ is a solution (note that it isn't "the" solution, but just "a" solution) to $\mathbf{X}^T\mathbf{X}\mathbf{w} = \mathbf{X}^T\mathbf{y}$.

Since $\mathbf{A} = \mathbf{X}^T\mathbf{X}$ is symmetric we can write it using its *eigenvalue decomposition* as $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^T$, where $\mathbf{D}$ is a diagonal matrix and $\mathbf{V}$ is an orthonormal matrix (that is, $\mathbf{V}^T\mathbf{V}$ is the identity $d \times d$ matrix). Define $\mathbf{D}^+$ to be the diagonal matrix³ such that

$$\mathbf{D}_{i,i}^+ = \begin{cases} 0 & \mathbf{D}_{i,i} = 0 \\ 1/\mathbf{D}_{i,i} & \text{otherwise.} \end{cases}$$

Now we define $\mathbf{A}^+ = \mathbf{V}\mathbf{D}^+\mathbf{V}^T$. We can prove that $\mathbf{A}\mathbf{A}^+\mathbf{A} = \mathbf{A}$:

³ $\mathbf{D}^+$ is "in essence the inverse of $\mathbf{D}$ ".

Proof.

$$\begin{aligned}
 \mathbf{A}\mathbf{A}^+\mathbf{A} &= \mathbf{A}(\mathbf{V}\mathbf{D}^+\mathbf{V}^T)\mathbf{A} \\
 &= \mathbf{V}\underbrace{\mathbf{D}\mathbf{V}^T\mathbf{V}}_I\mathbf{D}^+\underbrace{\mathbf{V}^T\mathbf{V}}_I\mathbf{D}\mathbf{V}^T \\
 &= \mathbf{V}\underbrace{\mathbf{D}\mathbf{D}^+}_I\mathbf{D}\mathbf{V}^T \\
 &= \mathbf{V}\mathbf{D}\mathbf{V}^T = \mathbf{A}
 \end{aligned}$$

$\Rightarrow \mathbf{A}^+$ is the generalized inverse of $\mathbf{A}$. □

So, if $\mathbf{X}^T\mathbf{X}$ is not invertible, we can find the model $\hat{\mathbf{w}} = \mathbf{A}^+\mathbf{X}^T\mathbf{y}$ that minimizes the $RSS(\mathbf{w})$ function, where $\mathbf{A}^+$ is the generalized matrix of $\mathbf{A} = \mathbf{X}^T\mathbf{X}$.

3.3.2 Linear regression for polynomial regression tasks

Some learning tasks call for nonlinear predictors, such as polynomial predictors. We can use as hypothesis set the set of polynomials of degree r with the tools that we have already developed for linear regression.

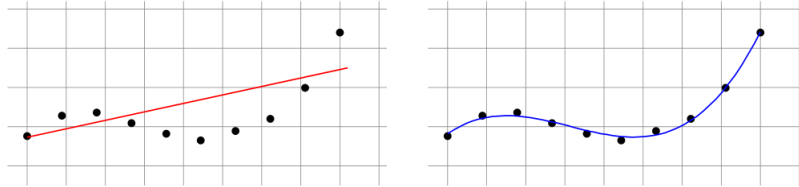


Figure 3.8

Let us assume $X = \mathbb{R}$, a polynomials of degree r can be written as $\sum_{i=0}^r w_i x^i = w_0 + w_1x + w_2x^2 + \dots + w_rx^r$. Given $x \in \mathbb{R}$ we can compute the following vector:

$$\mathbf{x}' = [1, x, x^2, \dots, x^r]^T, \quad \mathbf{w} = [w_0, w_1, w_2, \dots, w_r]^T$$

$$\langle \mathbf{w}, \mathbf{x}' \rangle = w_0 + w_1x + w_2x^2 + \dots + w_rx^r.$$

Thus, the hypothesis class of linear models for $\mathbf{x}'$ corresponds to the hypothesis class of polynomials of degree r with respect to x .

In the general case we have $X = \mathbb{R}^d$ so $\mathbf{x} = [x_1, x_2, \dots, x_d]$. In this case we use a technique called

feature expansion. For example let

$$\mathbf{x}' = [1, x_1, x_2, \dots, x_d, x_1^2, x_2^2, \dots, x_d^2, \dots, x_1^r, x_2^r, \dots, x_d^r]^T, \quad \mathbf{w} = [w_0, w_1, \dots, w_{r \cdot d}]^T.$$

We can find the optimal vector of coefficients $\mathbf{w}$ by using the Least Squares algorithm as shown for linear regression problems. We can define others feature expansion, for example: let assume $\mathbf{x} \in X = \mathbb{R}^3$, $\mathbf{x} = [x_1, x_2, x_3]$ and $r = 2$. We can use the following feature expansion:

$$\mathbf{x}' = [1, x_1, x_2, x_3, x_1^2, x_2^2, x_3^2, x_1x_2, x_1x_3, x_2x_3]^T, \quad \mathbf{w} = [w_0, w_1, \dots, w_9]^T$$

and then we build a linear model with respect to $\mathbf{x}'$.

To summarize: we can use the Least Squares algorithm to learn both linear and polynomial hypothesis. In the polynomial case we only have to expand the feature vector $\mathbf{x}$ in $\mathbf{x}'$ using a specific feature expansion.

3.4 Logistic regression

The goal of logistic regression is to find a function $h : \mathbb{R}^d \rightarrow [0, 1]$. Such hypotheses are useful for classification tasks, for instance in binary classification we have that $Y = \{\pm 1\}$ and $h(\mathbf{x})$ indicates the probability that label of $\mathbf{x}$ is 1. In the next we will focus on binary classification but similar consideration can be consider for multiclass classification.

3.4.1 Model

In logistic regression, the hypothesis class is defined as $H : \phi_{sig} \circ L_d$ where $\phi_{sig} : \mathbb{R} \rightarrow [0, 1]$ is the **sigmoid function**. To be more precise, in logistic regression we use a specific formulation of the sigmoid function which is called **logistic function**:

$$\phi_{sig}(z) = \frac{1}{1 + e^{-z}}. \quad (3.6)$$

As we can see, if $z \rightarrow +\infty$ then $\phi_{sig}(z) = 1$; if $z \rightarrow -\infty$ then $\phi_{sig}(z) = 0$. Therefore the hypothesis class H is defined as follows:

$$H_{sig} = \phi_{sig} \circ L_d = \{\mathbf{x} \rightarrow \phi_{sig}(\langle \mathbf{w}, \mathbf{x} \rangle) : \mathbf{w} \in \mathbb{R}^{d+1}\}$$

where $h_{\mathbf{w}}(\mathbf{x}) \in H_{sig}$ is defined as $h_{\mathbf{w}}(\mathbf{x}) = \frac{1}{1+e^{-\langle \mathbf{w}, \mathbf{x} \rangle}}$.

The main difference with binary classification with halfspaces is that, when the value $\langle \mathbf{w}, \mathbf{x} \rangle \approx 0$, then:

- the halfspace prediction is deterministically 1 or -1 ;
- $\phi_{sig}(\langle \mathbf{w}, \mathbf{x} \rangle) \approx 0.5$, i.e., the value can be either 0 or 1 with the same probability (this represent uncertainty in the predicted label).

3.4.2 Loss function

The loss function for logistic regression is defined as follows:

$$l(h_{\mathbf{w}}, (\mathbf{x}, y)) = \log(1 + e^{-y\langle \mathbf{w}, \mathbf{x} \rangle}) \quad (3.7)$$

Therefore, given training set $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ the ERM problem for logistic regression is defined as:

$$\arg \min_{\mathbf{w} \in \mathbb{R}^d} \frac{1}{m} \sum_{i=1}^m \log(1 + e^{-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle}) \quad (3.8)$$

The ERM problem can be solved efficiently since the loss function is a convex function.

Chapter 4

The Bias-Complexity Tradeoff

Given a training set S and a loss function $l(h, z)$, $z \in S$, our goal is to find a function $\hat{h}$ such that the generalization error $L_D(\hat{h})$ is small. To do so, we pick a learning algorithm A that, given S , will produce in output $\hat{h} = A(S)$. The choice of the algorithm A comprises: the hypothesis set H and the procedure to pick such $\hat{h} = A(S)$ from H . For example, if we choose the Perceptron algorithm, we are also choosing the linear functions as hypothesis class and a specific $\hat{h}$ that is returned in output.

But why do we need to make a choice? Is there a universal learner, i.e., an implementable algorithm A that predicts the best $\hat{h}$ for any distribution D ? The No-Free-Lunch theorem states that no such universal learner exists.

4.1 No-Free-Lunch Theorem

Theorem 12 (No-Free-Lunch). *Let A be any learning algorithm for the task of binary classification with respect to the 0 – 1 loss over a domain X . Let m be any number smaller than $|X|/2$, representing a training set size. Then, there exists a distribution D over $X \times \{0, 1\}$ (the relation between X and Y) such that:*

1. *There exists a function $f : X \rightarrow \{0, 1\}$ with $L_D(f) = 0$.*
2. *With probability of at least $1/7$ over the choice of $S \sim D^m$ we have that $L_D(A(S)) \geq 1/8$.*

This theorem states that for every learner, there exists a task on which it fails, even though that task can be successfully learned by another learner. Indeed, a trivial successful learner in this case would be an ERM learner with the hypothesis class $H = \{f\}$.

4.1.1 No Free Lunch and Prior Knowledge

Corollary 4. *Let X be an infinite domain set and let H be the set of all functions (i.e. not only linear or polynomial but all possible functions) from X to $\{0, 1\}$. Then, H is not PAC learnable.*

In general, every possible function from the domain to the label set is considered a good candidate¹. However the corollary above states that we will fail in finding an hypothesis using the ERM. How can we prevent such failures?

We need to use our prior knowledge about D to pick a good hypothesis set! But how should we choose² a good hypothesis class H ?

- We would like H to be large so that it may contain a function h with small $L_D(h)$.
- However the No-Free-Lunch tells us that H cannot be too large.

The sweet spot is a trade-off between the two.

4.2 Error decomposition

Let h_S be an ERM_H hypothesis. Then we can write:

$$L_D(h_S) = \underbrace{L_D(h_S) - \min_{h \in H} L_D(h)}_{\epsilon_{est} \geq 0} + \underbrace{\min_{h \in H} L_D(h)}_{\epsilon_{app} \geq 0} = \epsilon_{est} + \epsilon_{app}$$

where: ϵ_{est} is called **estimation error**; ϵ_{app} is called **approximation error**. Note that:

- Estimation error:
 - Derives from our inability to choose (with ERM) the best hypothesis (i.e. if $\epsilon_{est} = 0 \Rightarrow$ we have found the best hypothesis in the hypothesis class that we have considered).
 - In order to decrease his value we need a "small" hypothesis class H (more chances to find the best hypothesis on the selected class).
- Approximation error:
 - It doesn't depend on the training set but only on the hypothesis class H that we have considered: once we have chosen H , ϵ_{app} is unavoidable.

¹The class of hypothesis H of all the functions from X to $\{0, 1\}$ represents a lack of prior knowledge.

²General approach: we start from the simplest hypothesis class then we slightly increase the complexity until we find a good hypothesis.

- In order to decrease this value we need to increase the set of the hypothesis class (more chance to find an hypothesis with small generalization error).

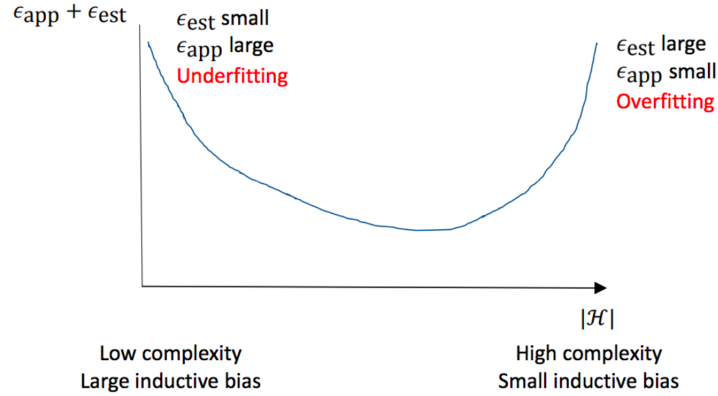


Figure 4.1

Since our goal is to minimize the total risk, we face a tradeoff, called the **bias-complexity tradeoff**. On one hand, choosing H to be a very rich class decreases the approximation error but at the same time might increase the estimation error, as a rich H might lead to **overfitting**. On the other hand, choosing H to be a very small set reduces the estimation error but might increase the approximation error or, in other words, might lead to **underfitting**.

4.3 Estimating the L_D

How can we estimate the generalization error $L_D(h)$ for a function h , for example for an hypothesis $h_S \in ERM_H$? We can use a **test set**, that is, a new set of samples not used for picking the hypothesis h_S . In practice we have one set of samples that we split in *training set* and *test set*. The instances in the test set must not be looked until we have picked our **final** hypothesis.

Chapter 5

Model Selection and Validation

When approaching some practical problem, we usually can think of several algorithms that may yield a good solution, each of which might have several parameters. How can we choose the best algorithm for the particular problem at hand? And how do we set the algorithm's parameters? This task is often called **model selection**.

Consider a regression task in which $X = \mathbb{R}$ and as always $Y = \mathbb{R}$. We want to learn a one dimensional function $h : \mathbb{R} \rightarrow \mathbb{R}$. Suppose that our training set has the following samples:

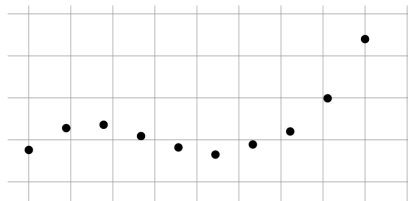


Figure 5.1

By looking at the data we can say that choosing H to be the class of polynomials functions is a good idea. To learn the model we can use the linear regression machinery that we have seen in Subsection 3.3.2. However, we might be uncertain regarding which degree d would give the best results for our data set: a small degree may not fit the data well (i.e., it will have a large approximation error), whereas a high degree may lead to *overfitting* (i.e., it will have a large estimation error).

What about considering the empirical risk of best hypothesis of various degrees? For example lets consider $d = 2, 3, 10$:

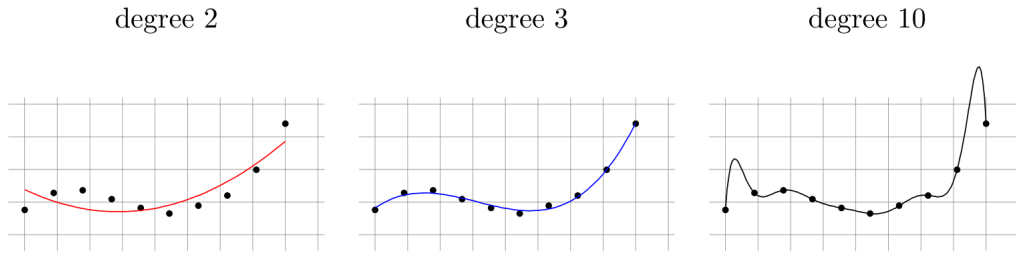


Figure 5.2: Best hypothesis for degree $d = \{2, 3, 10\}$. Intuition: degree 10 offers the lowest training error but it suffers from overfitting problem. Degree 3 seems to be the best option.

5.1 Validation

5.1.1 Hold Out Set

Assume that we have picked a predictor h (e.g., by ERM rule on the hypothesis class¹ H_d). We can use new data to *estimate the true error* of h . This new data forms the **validation set** V .

Let $V = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{m_v}, y_{m_v})\}$ be a set of m_v fresh samples (not used in the training set) from the distribution D and let

$$L_V(h) = \frac{1}{m_v} \sum_{i=1}^{m_v} l(h, (\mathbf{x}_i, y_i)) \quad (5.1)$$

be the validation error.

Theorem 13. *Let h be some predictor and assume that the loss function is in $[0, 1]$. Then for every $\delta \in]0, 1[$, with probability of at least $1 - \delta$ over the choice of a validation set V of size m_v we have*

$$|L_V(h) - L_D(h)| \leq \sqrt{\frac{\log(2/\delta)}{2m_v}}$$

The bound above does not depend on the algorithm or the training set used to construct h .

In practice its not possible to use "fresh" data, what we can do is: split our dataset in training and validation set (often called hold out set).

5.1.2 Validation for Model Selection

Validation can be naturally used for model selection² as follows. Assume that we have $H = \bigcup_{i=1}^r H_i$ (for example $H_i :=$ set of polynomials of degree i). Given a training set S , let h_i be the

¹For example H_d can be the class of hypotheses of degree d .

²Choosing the model means either choosing the hypothesis, the class of hypothesis or a parameter of the algorithm.

hypothesis obtained by ERM rule from H_i using S . Now, how do we pick the final hypothesis from $\{h_1, \dots, h_r\}$? We can use the validation set!

Let $V = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_{m_v}, y_{m_v})\}$ be the validation set composed by m_v samples taken from D . We choose as *final hypothesis* (or class or value of parameter) from $\{h_1, \dots, h_r\}$ by ERM over the validation set.

Theorem 14. *Let $H = \{h_1, \dots, h_r\}$ be an arbitrary set of predictors and assume that the loss function is in $[0, 1]$. Assume that a validation set V of size m_v is sampled independent of H . Then for every $\delta \in]0, 1[$, with probability of at least $1 - \delta$ over the choice of V we have*

$$\forall h \in H : |L_D(h) - L_V(h)| \leq \sqrt{\frac{\log(2|H|/\delta)}{2m_v}}$$

For instance consider the example that we have introduced at the beginning of this chapter. We have found, using the ERM for H_1, H_2, H_3 , that:

$h_1 :=$ best polynomial of degree 2 hypothesis;

$h_2 :=$ best polynomial of degree 3 hypothesis;

$h_3 :=$ best polynomial of degree 10 hypothesis.

Which model is the best one? We pick one hypothesis over $\{h_1, h_2, h_3\}$ by ERM over the validation set!

5.1.3 Model-Selection Curve

The model selection curve shows the training error and validation error as functions of the complexity of the model considered.

From the Figure 5.3 we can observe that:

- The training error decrease monotonically as we increase the complexity of the model (i.e. the degree of the polynomial).
- Until degree 3, the validation error is lower than the training error, this can happened: remember that the validation error is only an estimation of the generalization error. Usually $L_S(h)$ is lower than $L_V(h)$ because the ERM choose the hypothesis by minimizing the training error without considering the validation error.
- At first the training error and the validation error are very high $\Rightarrow$ *underfitting*. As the complexity grows, the validation error increases $\Rightarrow$ *overfitting*.

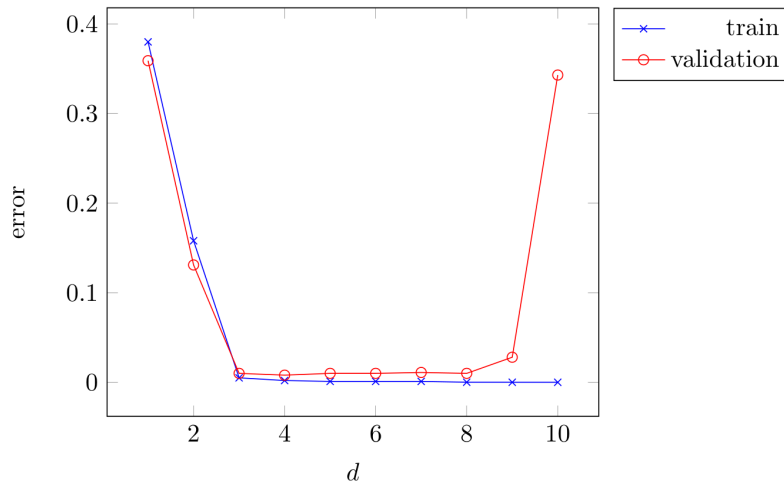


Figure 5.3: Model selection curve for the polynomial fitting problem.

Plotting such curves can help us understand whether we are searching the correct regime of our parameter space.

What we can do when we have more parameters with values in $\mathbb{R}$? We can do the following:

1. Start with a rough grid of value. For example consider the parameter $d \in \{2, 3, 10\}$ (degree of the polynomial) and another parameter $\lambda \in \mathbb{R}$. We have to choose some value of λ , e.g $\lambda \in \{4, 10, 20\}$, then we learn a model for every pair of parameter $d \times \lambda$.
2. Plot the corresponding model-selection curve.
3. Based on the curve, we zoom into the correct regime.
4. Restart from point (1) using a finer grid (i.e. change the possible value of λ or d based on the correct regime).

Note that: the empirical risk on the validation set is not an estimate of the true risk, in particular if we choose among many models (i.e. if r is big). This is the same thing that happen when we choose the hypothesis with ERM on the training set: the hypothesis that we choose is the one that performs well on that data!

Now that we have defined a method to find the model, how do we estimate the true risk? We need fresh data for the test set!

5.1.4 Train-Validation-Test split

Assume that we have $H = \bigcup_{i=1}^r H_i$. We have to split our dataset in 3 parts:

1. **Training set:** used to learn the best model h_i from each H_i .
2. **Validation set:** used to pick one hypothesis h from $\{h_1, \dots, h_r\}$ (i.e. h is the best model).
3. **Test set:** used to estimate the true risk $L_D(h)$.

The estimate from the test set has the guarantees provided by the theorem 13 on estimate of $L_D(h)$ (for one class of hypothesis)³.

Note that the test set is not involved in the choice of h ! If after using the test set to estimate $L_D(h)$ we decide to choose another hypothesis, we cannot use the test set again to estimate $L_D(h)$!

5.1.5 k-Fold Cross Validation

The validation procedure described so far assumes that data is plentiful and that we have the ability to sample a fresh validation set. But in some applications, data is scarce and we do not want to "waste" data on validation. The k -fold cross validation technique is designed to give an *accurate estimate of the true error* without wasting too much data.

The procedure is the following:

1. Partition the training set into k folds of size m/k .
2. For each fold:
 - (a) Train on union of other folds.
 - (b) Estimate error (for learned hypothesis) from the fold (= current validation set).
3. The estimate of the true error is obtained as the average of the estimated errors computed above.

A special case, called *leave-one-out* is when $k = m$, where m is the number of examples: at each iteration the validation set has only one sample. This approach is great in theory but it requires too much time in practice (needs to learn $k = m$ hypotheses).

³So, in the theorem, L_V is the test error not the validation error in our case.

5.1.6 k-Fold Cross Validation for Model Selection

k-Fold cross validation is often used for model selection (or parameter tuning), and once the best parameter is chosen, the algorithm is retrained using this parameter on the entire training set.

Algorithm 5 *k*-Fold Cross Validation for Model Selection

Require: Training set $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$, set of parameter values Θ , learning algorithm A , integer k

- 1: Partition S into $S_1, S_2, \dots, S_k$ ▷ The k -Folds
 - 2: **for all** $\theta \in \Theta$ **do** ▷ For each parameter
 - 3: **for** $i = 1 \dots k$ **do** ▷ For each fold
 - 4: $h_{i,\theta} = A(S \setminus S_i, \theta)$ ▷ Find the best hypothesis excluding fold i and using parameter θ
 - 5: **end for**
 - 6: $\text{error}(\theta) = \frac{1}{k} \sum_{i=1}^k L_{S_i}(h_{i,\theta})$ ▷ Estimate the true error using the average of the losses (on the validation sets used in the for loop above)
 - 7: **end for**
 - 8: $\theta^* = \arg \min_{\theta \in \Theta} \text{error}(\theta)$ ▷ Find the parameter that provides the best model, i.e. the one that has the lowest (estimate of the) true error
 - 9: **return** $h_{\theta^*} = A(S, \theta^*)$ ▷ Learn the best hypothesis with the best parameter, using all the training data
-

5.2 What to do if learning fails?

What can we do if the results on the test set are bad? First of all we need to understand where the error comes from.

5.2.1 $L_S(h_S)$ is large: underfitting

Let $h^* \in \arg \min_{h \in H} L_D(h)$, i.e. h^* is the best hypothesis of the hypotheses class H that we're considering. The training error or h_S can be decomposed like the following:

$$L_S(h_S) = (L_S(h_S) - L_S(h^*)) + (L_S(h^*) - L_D(h^*)) + L_D(h^*)$$

where:

- $(L_S(h_S) - L_S(h^*)) \leq 0$ because $h_S \in \arg \min_{h \in H} L_S(h)$
- $(L_S(h^*) - L_D(h^*)) \approx 0$

Therefore $L_S(h_S)$ is large because $L_D(h^*)$ is large, that is, the approximation error is large!

The problem in this case is the hypotheses class H , we need to change/enlarge it.

5.2.2 $L_S(h_S)$ is small

In this case we need to understand if $L_D(h^*)$ is large or not. But how can we do that? We can plot the **learning curves**! To produce a learning curve we train the algorithm on prefixes of the data of increasing sizes, until we reach the maximum value of examples (i.e. m).

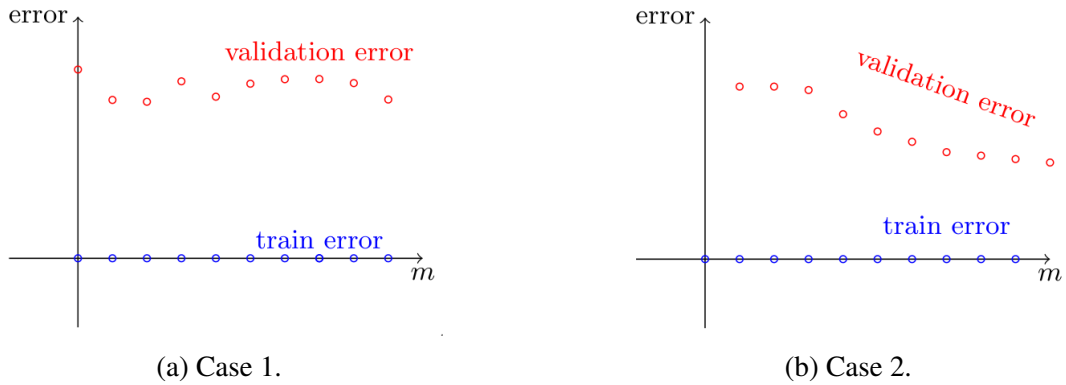


Figure 5.4: The training error is low for every value of the prefixes. This is because $L_S(h_S)$ is low (on the whole training set), so there exists for sure an hypothesis that has a low training error when we train the model considering a training set with a number of examples $< m$.

Case 1: the validation error (i.e. the approximation of the generalized error) is almost constant, so there is no evidence that the approximation error of H is small. In this case we need to change or enlarge the class H .

Case 2: the validation error is decreasing, so H may have a good approximation error but maybe we do not have enough data.

5.2.3 Main approaches

Some potential steps to follow if learning fails:

- if you have parameters to tune, plot model-selection curve to make sure they are tuned appropriately
- if training error is excessively large consider:
 - enlarge H ;
 - change H ;
 - change feature representation of the data (it can help);

- if training error is small, use learning curves to understand whether problem is *approximation error* (or *estimation error*)
 - if approximation error seems small (case 2):
 - * get more data;
 - * reduce complexity of H ;
 - if approximation error seems large (case 1):
 - * change H ;
 - * change feature representation of the data.

Chapter 6

Regularization and feature selection

So far we know that, in order to find a good hypothesis that predicts well our distribution D , we have to: define an algorithm A that, given the training set S , produces the hypothesis $\hat{h} = A(S)$ such that the empirical risk $L_S(\hat{h})$ is small, or at least close to the smallest generalization error $L_D(h^*)$ achievable by the "best" hypothesis $h^* \in H$, i.e.: $h^* = \arg \min_{h \in H} L_D(h)$.

In the first chapter we have seen a *learning paradigm* called *Empirical Risk Minimization* where the choice was $\hat{h} = A(S) \in \text{ERM}_H(S)$. In the following sections we are going to introduce another learning paradigm.

6.1 Regularized Loss minimization

Assume h is defined by a vector $\mathbf{w} = (w_1, \dots, w_d)^T \in \mathbb{R}^d$ (e.g., linear models). We define the **regularization function** as a function $R : \mathbb{R}^d \rightarrow \mathbb{R}$. The **Regularized Loss Minimization (RLM)** is defined as follows: pick h obtained as

$$\arg \min_{\mathbf{w}} (L_S(\mathbf{w}) + R(\mathbf{w})). \quad (6.1)$$

The regularization $R(\mathbf{w})$ is a "measure of complexity" of hypothesis h defined by $\mathbf{w}$. The algorithm balances between low empirical risk and "simpler", or "less complex", hypotheses.

But how we do actually define the regularization function R ? We will see some of the most common regularization function.

6.2 ℓ_1 Regularization

The regularization is defined as $R(\mathbf{w}) = \lambda \|\mathbf{w}\|_1$, where

- $\lambda \in \mathbb{R}, \lambda > 0$
- ℓ_1 norm: $\|\mathbf{w}\|_1 = \sum_{i=1}^d |w_i|$

Therefore the learning rule is to pick

$$A(S) = \arg \min_{\mathbf{w}} (L_S(\mathbf{w}) + \lambda \|\mathbf{w}\|_1).$$

The value $\|\mathbf{w}\|_1$ measures the "complexity" of hypothesis defined by $\mathbf{w}$, instead λ regulates the tradeoff between the empirical risk $L_S(\mathbf{w})$ or overfitting and complexity $\|\mathbf{w}\|_1$ of the model we pick. For instance, if λ is huge, then all data in S are ignored and the algorithm choose the model with $\mathbf{w} = \mathbf{0}$ (the simplest one).

6.2.1 LASSO = linear regression + ℓ_1 regularization

Using the LASSO (*Least Absolute Shrinkage and Selection Operator*) means solving a linear regression problem by using the squared loss function and the ℓ_1 regularization. The lasso pick

$$\mathbf{w} = \arg \min_{\mathbf{w}} \left(\lambda \|\mathbf{w}\|_1 + \sum_{i=1}^m (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)^2 \right) \quad (6.2)$$

the hypothesis h defined by $\mathbf{w}$ that minimizes the sum between the Residual Sum of Squares (RSS) and the ℓ_1 norm.

6.3 Tikhonov regularization

The regularization function is defined as $R(\mathbf{w}) = \lambda \|\mathbf{w}\|_2$, where

- $\lambda \in \mathbb{R}, \lambda > 0$
- ℓ_2 norm: $\|\mathbf{w}\|_2 = \sqrt{\sum_{i=1}^d w_i^2}$

Therefore the learning rule is to pick

$$A(S) = \arg \min_{\mathbf{w}} (L_S(\mathbf{w}) + \lambda \|\mathbf{w}\|_2).$$

The value $\|\mathbf{w}\|_2$ measures the "complexity" of hypothesis defined by $\mathbf{w}$, instead λ regulates the tradeoff between the empirical risk $L_S(\mathbf{w})$ or overfitting and complexity $\|\mathbf{w}\|_2$ of the model we pick. The Tikhonov regularization is the one that is the most used in practice.

6.3.1 Ridge Regression = linear regression + Tikhonov regularization

Using the Ridge Regression means solving a linear regression problem by using the squared loss function and the Tikhonov regularization. The Ridge Regression pick

$$\mathbf{w} = \arg \min_{\mathbf{w}} \left(\lambda \|\mathbf{w}\|_2 + \sum_{i=1}^m (\langle \mathbf{w}, \mathbf{x}_i \rangle - y_i)^2 \right) \quad (6.3)$$

the hypothesis h defined by $\mathbf{w}$ that minimizes the sum between the Residual Sum of Squares (RSS) and the ℓ_2 norm.

Compute the value of $\mathbf{w}$ using the matrix notation In subsection 3.3.1, we can rewrite the RSS using the matrix notation. Using that notation, we can rewrite the 6.3 as follows:

$$\mathbf{w} = \arg \min_{\mathbf{w}} (\lambda \|\mathbf{w}\|^2 + (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})).$$

In order to find the value of $\mathbf{w}$ we can do the following:

$$f(\mathbf{w}) = \lambda \|\mathbf{w}\|^2 + (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

compute the derivative and compare it to 0:

$$\frac{\partial f(\mathbf{w})}{\partial \mathbf{w}} = 2\lambda \mathbf{w} - 2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$2\lambda \mathbf{w} - 2\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w}) = 0 \quad \Leftrightarrow \quad (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X}) \mathbf{w} = \mathbf{X}^T \mathbf{y}$$

Note that: $\mathbf{X}^T \mathbf{X}$ is positive semidefinite, $\lambda \mathbf{I}$ is positive semidefinite, so $\Rightarrow (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})$ is positive semidefinite $\Rightarrow (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})$ is invertible!

The Ridge regression solution is computed as follows:

$$\mathbf{w} = (\lambda \mathbf{I} + \mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (6.4)$$

6.4 Feature selection

In general, in machine learning one has to decide what to use as features (i.e. input) for learning. Even if somebody gives us a representation as a feature vector, maybe there is a "better" representation. But what does it mean "better"?

Consider for example the features x_1, x_2 and the output y . Let

$$\begin{aligned} x_1 &\sim \text{Uniform}(-1, 1) \\ y &= x_1^2 \\ x_2 &\sim y + \text{Uniform}(-0.001, 0.001) \end{aligned}$$

If we want to predict y , which feature is better: x_1 or x_2 ?

- x_1 because y is a deterministic function of x_1 . However, we need to consider x_1^2 as the feature; a linear model with x_1 as feature, don't work well.
- x_2 : because with linear models we predict y exactly up to noise, and the noise is low (i.e. predicting y is a good predictor). However we can do better by using x_1^2 as a feature.

The answer could be: take all the possible feature. But the *No-Free-Lunch Theorem* tells us that our learner is going to fail if we do so $\Rightarrow$ we have to make a choice.

Let us consider the following scenario: we have a large pool of features and our goal is to select a small number of features that will be used by our (final) predictor. Assume that we have $X = \mathbb{R}^d$ (d features) and we want to learn a predictor that only relies on $k \ll d$ features. Why this is important?

- Constraining the hypothesis class to use a small subset of features can reduce its estimation error and thus prevent overfitting.
- Prediction can be done faster.
- In applications such as medical diagnostics, obtaining each possible "feature" (e.g., test result) can be costly.

6.4.1 Subset Selection algorithm

Assume that we use the Empirical Risk minimization procedure and assume that an hypothesis $h \in H$ corresponds to a vector of weights $\mathbf{w} \in \mathbb{R}^d$. The problem of selecting k features that minimize the Empirical Risk can be rewritten as

$$\min_{\mathbf{w}} L_S(\mathbf{w}) \quad \text{subject to} \quad \|\mathbf{w}\|_0 \leq k$$

where $\|\mathbf{w}\|_0 = \{i : w_i \neq 0\}$. How can we solve this problem? Well, ideally can try all subsets of k out of d features and choose the subset which leads to the best performing predictor.

Algorithm 6 Subset selection algorithm

```

1: Let  $I = \{1, \dots, d\}$  ▷ Features indices
2: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
3:  $P^{(k)} \leftarrow \{J \subseteq I : |J| = k\}$  ▷ All the possible subsets of  $k$  elements over  $d$ .
4: for  $p \in P^{(k)}$  do ▷ For each subset of  $k$  elements
5:    $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn a model using only  $k$  features
6: end for
7: return  $p_k \leftarrow \arg \min_{p \in P^{(k)}} L_S(h_p)$  ▷ Return the set of features that provide the lowest training error (computed in the for loop above)

```

The Algorithm 6 implements this idea. The problem of the algorithm is the complexity: the computation of all the possible subset of k elements over d are $\Theta\left(\binom{d}{k}\right) \in \Theta(d^k)$, so we need to learn a exponential number of models! Unfortunately we cannot do better than this because of the following proposition:

Proposition 4. *The Optimization problem of features selection is NP-hard.*

Where *NP* stands for *non deterministic polynomial*. In other words means that it is unlikely that there is a polynomial algorithm that solves our problem.

The only left possibility is to consider **greedy algorithms**! While these methods cannot guarantee finding the optimal subset, they often work reasonably well in practice.

6.4.2 Forward Selection algorithm

The Forward Selection algorithm starts from the empty set of indices and adds, at each iteration, one index (i.e. one feature) at the time, until the set has cardinality of k .

The algorithm complexity: $\sum_{i=0}^{k-1} 1(\sum_{j=0}^{d-i-1} 1) = \Theta(kd)$. So the algorithm learns a total of $\Theta(kd)$ models.

6.4.3 Backward Selection algorithm

The Backward Selection algorithm starts from a copy of the I set and removes, at each iteration, one index (i.e. one feature) at the time, until the set has cardinality of k .

The algorithm complexity: $\sum_{i=0}^{d-k-1} 1(\sum_{j=0}^{d-i-1} 1) = \Theta((d-k)d)$. So the algorithm learns a total of $\Theta((d-k)d)$ models.

Algorithm 7 Forward Selection algorithm

```

1: Let  $I = \{1, \dots, d\}$  ▷ Features indices
2: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
3:  $sol \leftarrow \emptyset$  ▷ Chosen indices (= features)
4: while  $|sol| < k$  do
5:   for  $i \in I \setminus sol$  do ▷ Consider all the non chosen indices
6:      $p \leftarrow sol \cup \{i\}$  ▷ Add a one index
7:      $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn  $h_p$  using only the indices in  $p$ 
8:   end for
9:    $sol \leftarrow sol \cup \arg \min_{i \in I \setminus sol} L_S(h_{sol \cup \{i\}})$  ▷ Add to the solution the index  $i$  that has provided the lowest training error (computed in the for loop above)
10: end while
11: return  $sol$ 

```

Algorithm 8 Backward Selection algorithm

```

1: Let  $I = \{1, \dots, d\}$  ▷ Features indices
2: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
3:  $sol \leftarrow \emptyset \cup I$  ▷ Chosen indices (= features)
4: while  $|sol| > k$  do
5:   for  $i \in sol$  do ▷ Consider all the indices of the current solution
6:      $p \leftarrow sol \setminus \{i\}$  ▷ Remove one index
7:      $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn  $h_p$  using only the indices in  $p$ 
8:   end for
9:    $sol \leftarrow sol \setminus \arg \min_{i \in sol} L_S(h_{sol \setminus \{i\}})$  ▷ Remove from the current solution the index  $i$  that has provided the lowest training error (computed in the for loop above)
10: end while
11: return  $sol$ 

```

6.5 Subset, Forward, Backward Selection with Validation Data

In the previous subsections we have used only the training set to select the best hypothesis and we know that this could lead us to overfitting. So the best choice is to use the validation (or cross validation): split data into training data and validation data, learn models on training, evaluate (pick among different hypothesis models) on validation. We can modify the algorithms seen in the previous sections.

With this algorithm we can actually get as best model (in terms of validation error) a model with less than k features. Using only the training set to do the same operation wouldn't make any sense.

Algorithm 9 Subset selection algorithm

```

1: Let  $S$  be the training set
2: Let  $V$  be the validation set
3: Let  $I = \{1, \dots, d\}$  ▷ Features indices
4: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
5: for  $l \leftarrow 0$  to  $k$  do
6:    $P^{(l)} \leftarrow \{J \subseteq I : |J| = l\}$  ▷ All the possible subsets of  $l$  elements over  $d$ .
7:   for  $p \in P^{(l)}$  do ▷ For each subset of  $l$  elements
8:      $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn a model using only  $l$  features
9:   end for
10:   $p_l \leftarrow \arg \min_{p \in P^{(l)}} L_V(h_p)$  ▷ Set of  $l$  features that provide the lowest validation error
11: end for
12: return  $\arg \min_{p \in \{p_0, \dots, p_k\}} L_V(h_p)$  ▷ Return the set of features that provide the lowest validation error (computed in the for loop above)

```

Algorithm 10 Forward Selection algorithm

```

1: Let  $S$  be the training set
2: Let  $V$  be the validation set
3: Let  $I = \{1, \dots, d\}$  ▷ Features indices
4: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
5:  $sol \leftarrow \emptyset$  ▷ Chosen indices (= features)
6: while  $|sol| < k$  do
7:   for  $i \in I \setminus sol$  do ▷ Consider all the non chosen indices
8:      $p \leftarrow sol \cup \{i\}$  ▷ Add a one index
9:      $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn  $h_p$  using only the indices in  $p$ 
10:   end for
11:   $sol \leftarrow sol \cup \arg \min_{i \in I \setminus sol} L_V(h_{sol \cup \{i\}})$  ▷ Add to the solution the index  $i$  that has provided the lowest validation error (computed in the for loop above)
12: end while
13: return  $sol$ 

```

Algorithm 11 Backward Selection algorithm

```

1: Let  $I = \{1, \dots, d\}$  ▷ Features indices
2: Define  $p = \{i_1, \dots, i_k\} \subseteq I$  ▷  $H_p$  contains hypotheses/models where only features  $w_{i_1}, \dots, w_{i_k}$  are used
3:  $sol \leftarrow \emptyset \cup I$  ▷ Chosen indices (= features)
4: while  $|sol| > k$  do
5:   for  $i \in sol$  do ▷ Consider all the indices of the current solution
6:      $p \leftarrow sol \setminus \{i\}$  ▷ Remove one index
7:      $h_p \leftarrow \arg \min_{h \in H_p} L_S(h)$  ▷ Learn  $h_p$  using only the indices in  $p$ 
8:   end for
9:    $sol \leftarrow sol \setminus \arg \min_{i \in sol} L_V(h_{sol \setminus \{i\}})$  ▷ Remove from the current solution the index  $i$  that has provided the lowest validation error (computed in the for loop above)
10: end while
11: return  $sol$ 

```

6.6 Subset, Forward, Backward Selection with cross-validation

If we don't have enough data for the validation set, we can perform the features selection using only the training data and the k-fold cross validation technique.

Chapter 7

Support Vector Machines

7.1 Introduction

7.1.1 Classification and margin

Consider a classification problem with two classes: the instance set $X = \mathbb{R}^d$ and the label set $Y = \{-1, 1\}$. Let $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ be the training set and H the class of halfspaces (linear models).

Let us also assume that the data are linearly separable, so there exist an halfspace that perfectly classifies the training set. The question is: in general, there exists multiple separating hyperplanes that correctly classifies all the training data, but which one is the best choice?

Informally, for a given separating halfspace we define its *margin* as its minimum distance to an example in the training set S . So, the best separating hyperplane is the one with largest margin, i.e. the one that identifies a "clear separating plane". But how can we compute such model?

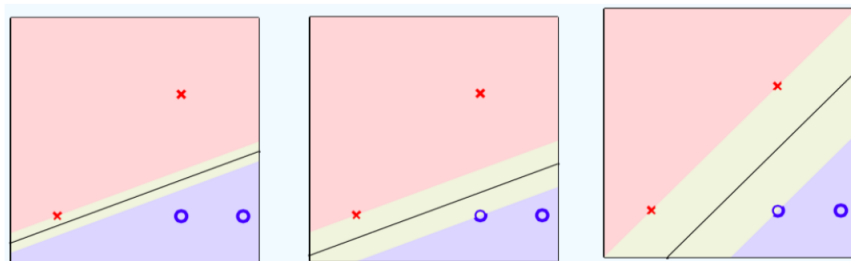


Figure 7.1: The last one seems the best choice, since it can tolerate more "noise".

7.1.2 Linearly separable training set

If the training set S is linearly separable (assumption), there exists an halfspace $(\mathbf{w}, b)$ such that $y_i = \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$ for all $i = 1, \dots, m$. In an equivalent way we can write

$$\forall i = 1, \dots, m : y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0.$$

Informally we say that the margin of a separating hyperplane is its minimum distance to an example in the training set S .

7.1.3 Separating Hyperplane and Margin

Given hyperplane defined by $L = \{\mathbf{v} : \langle \mathbf{w}, \mathbf{v} \rangle + b = 0\}$, and given a point $\mathbf{x}$, the distance of $\mathbf{x}$ to L is defined as

$$d(\mathbf{x}, L) = \min\{\|\mathbf{x} - \mathbf{v}\| : \mathbf{v} \in L\}.$$

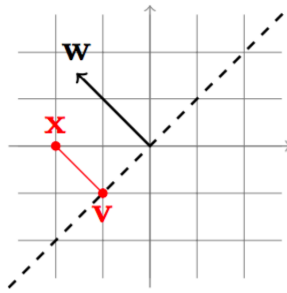


Figure 7.2

If $\|\mathbf{w}\| = 1$ then the computation of the distance is much simpler to get: $d(\mathbf{x}, L) = |\langle \mathbf{w}, \mathbf{x} \rangle + b|$. So in order to find the best separating hyperplane, we will search for a vector of weights $\mathbf{w}$ such that $\|\mathbf{w}\| = 1$.

7.1.4 Margin and Support Vectors

The margin of a separating hyperplane is the distance of the closest example in the training set to it. In general the margin is computed as follows:

$$\min_{i \in \{1, \dots, m\}} \frac{|\langle \mathbf{w}, \mathbf{x}_i \rangle + b|}{\|\mathbf{w}\|}$$

so if $\|\mathbf{w}\| = 1$ the margin is simply: $\min_{i \in \{1, \dots, m\}} |\langle \mathbf{w}, \mathbf{x}_i \rangle + b|$.

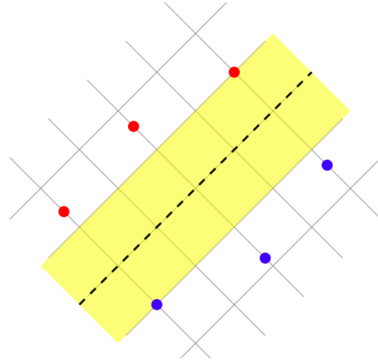


Figure 7.3: The closest examples are called support vectors.

7.2 Support Vector Machine: Hard-SVM

SVM is the approach that we use for linear models, in order to find the model that maximize the margin.

Under the assumption that the training set S is linearly separable, the approach of **Hard-SVM**, is used to find the separating hyperplane with largest margin.

$$(\hat{\mathbf{w}}, \hat{b}) = \arg \max_{(\mathbf{w}, b): \|\mathbf{w}\|=1} \overbrace{\{\min_{i \in \{1, \dots, m\}} |\langle \mathbf{w}, \mathbf{x}_i \rangle + b| \}}^{\text{def of margin}} \quad \text{s.t. } \forall i: y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0 \quad (7.1)$$

Due to separability assumption we can use the **equivalent formulation**:

$$(\hat{\mathbf{w}}, \hat{b}) = \arg \max_{(\mathbf{w}, b): \|\mathbf{w}\|=1} \{\min_{i \in \{1, \dots, m\}} y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)\} \quad (7.2)$$

because $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = |\langle \mathbf{w}, \mathbf{x}_i \rangle + b|$ if $y_i = \text{sign}(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$.

Another equivalent formulation is the **Quadratic Programming Formulation**:

Algorithm 12 Hard-SVM: quadratic programming formulation

• **Input:** $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

• **Solve:**

$$(\mathbf{w}_0, b_0) = \arg \min_{(\mathbf{w}, b)} \|\mathbf{w}\|^2 \quad \text{s.t. } \forall i: y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$$

• **Output:** (normalize)

$$\hat{\mathbf{w}} = \frac{\mathbf{w}_0}{\|\mathbf{w}_0\|}, \quad \hat{b} = \frac{b_0}{\|\mathbf{w}_0\|}$$

This is a quadratic optimization problem: the objective is a quadratic convex function and all the constraints are linear inequalities. There exists efficient quadratic programming solvers that can provide a solution for our problem.

Intuitively, hard-SVM searches for $\mathbf{w}$ of minimal norm among all the vectors that separate the data and for which $|\langle \mathbf{w}, \mathbf{x}_i \rangle + b| \geq 1$ for all i . In other words, *we enforce the margin to be 1*, but now the units in which we measure the margin scale with the norm of $\mathbf{w}$. Therefore, finding the largest margin halfspace boils down to finding $\mathbf{w}$ whose norm is minimal.

7.2.1 The homogeneous case and Support Vectors

It is often more convenient to consider homogenous halfspaces, namely, halfspaces that pass through the origin and are thus defined by $\text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle)$, where the bias term b is set to be zero.

$$\mathbf{w}_0 = \min_{\mathbf{w}} \|\mathbf{w}\|^2 \quad \text{s.t. } \forall i : y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \geq 1. \quad (7.3)$$

As we said earlier, the **support vectors** are the vectors at minimum distance from $\mathbf{w}_0$ (or equivalently from $\mathbf{w}_0$ normalized). The support vector are the only ones that matter for defining $\mathbf{w}_0$.

Proposition 5. *Let $\mathbf{w}_0$ be as above. Let $I = \{i : |\langle \mathbf{w}_0, \mathbf{x}_i \rangle| = 1\}$ (i.e. the support vectors are all the vectors $\mathbf{x}_i$ such that $i \in I$). Then exist coefficients $\alpha_1, \dots, \alpha_m$ such that*

$$\mathbf{w}_0 = \sum_{i \in I} \alpha_i \mathbf{x}_i.$$

Solving Hard-SVM is equivalent to find α_i for $i = 1, \dots, m$, and $\alpha_i \neq 0$ only for support vectors (the dual problem).

7.3 Support Vector Machine: Soft-SVM

Hard-SVM works only if the data are linearly separable. In general in do not know if the data are linearly separable so instead of using Hard-SVM, we use Soft-SVM. The idea of Soft-SVM is to modify the constraints of Hard-SVM to allow from some violation, but take into account violations into objective function.

In hard-SVM the constraints are: $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 \quad \forall i \in \{1, \dots, m\}$. In Soft-SVM we relax those constraints by adding a slack variable $\xi_1, \dots, \xi_m$ to each constraint. We get: $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \quad \forall i \in \{1, \dots, m\}$. The value of ξ_i indicates how much the constraint $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$ is violated.

Soft-SVM minimizes the combination of the norm of $\mathbf{w}$ and the average of the values of ξ_i $\forall i$ (i.e. the average of the total violation). The tradeoff between the two terms is controlled by a parameter $\lambda \in \mathbb{R}$, $\lambda > 0$.

Algorithm 13 Soft-SVM

• **Input:** $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

• **Parameter:** $\lambda > 0$

• **Solve:**

$$\min_{\mathbf{w}, b, \xi} \left(\lambda \|\mathbf{w}\|^2 + \frac{1}{m} \sum_{i=1}^m \xi_i \right)$$

$$\text{s.t. } \forall i, y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

• **Output:** $\mathbf{w}, b$

The objective function is similar to the one we have seen for the Regularized Loss Minimization paradigm (section 6.1) with the Tikhonov regularization function.

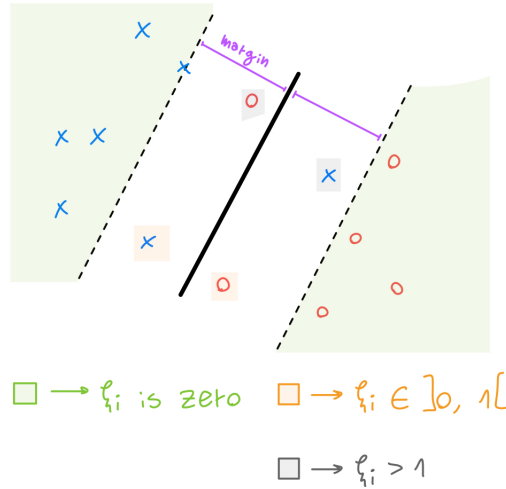


Figure 7.4: Soft-SVM: geometrical point of view.

We can define an **equivalent formulation** for the Soft-SVM. Consider the **hinge loss** defined as follows:

$$l^{\text{hinge}}((\mathbf{w}, b), (\mathbf{x}, y)) = \max\{0, 1 - y(\langle \mathbf{w}, \mathbf{x} \rangle + b)\} \quad (7.4)$$

where we are considering $(\mathbf{w}, b)$ the parameter that identifies our (linear) hypothesis. Given

$(\mathbf{w}, b)$ and a training set S , the empirical risk $L_S^{\text{hinge}}((\mathbf{w}, b))$ is defined as:

$$L_S^{\text{hinge}}((\mathbf{w}, b)) = \frac{1}{m} \sum_{i=1}^m l^{\text{hinge}}((\mathbf{w}, b), (\mathbf{x}_i, y_i)) \quad (7.5)$$

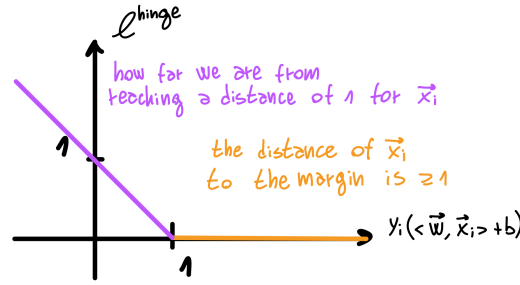


Figure 7.5

Algorithm 14 Soft-SVM Equivalent formulation

• **Input:** $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$

• **Parameter:** $\lambda > 0$

• **Solve:**

$$\min_{\mathbf{w}, b} \left(\lambda \|\mathbf{w}\|^2 + L_S^{\text{hinge}}(\mathbf{w}, b) \right)$$

• **Output:** $\mathbf{w}, b$

The algorithm above is actually the Regularized Loss Minimization with the Tikhonov regularization function (or ℓ_2 regularization).

Note that: Soft-SVM seeks linear models by minimizing the sum of the hinge loss function and the ℓ_2 regularization. In the subsection 3.2.3 we have seen that the SGD seeks linear models by using the ERM, and we can see that the loss function used in that chapter is very similar to the hinge loss, apart from the "1 –" at the beginning. Another difference between the two is that the Soft-SVM uses the Regularized Loss Minimization, the SGD uses the ERM. But this isn't a real difference because we can easily modify the SGD in order to introduce the regularization part.

7.4 Implementing Soft-SVM using SGD

In order to solve the Soft-SVM problem, we have two options: the first one is to use a standard solver for (quadratic) problems; the second one is to use the Stochastic Gradient Descent. Let us consider the second option. What we want to do is to find

$$\min_{\mathbf{w}, b} \left(\frac{\lambda}{2} \|\mathbf{w}\|^2 + \frac{1}{m} \sum_{i=1}^m \max\{0, 1 - y_i \langle \mathbf{w}, \mathbf{x}_i \rangle\} \right) \quad (7.6)$$

where we have added a $1/2$ in the regularization term in order to simplify some computation.

Algorithm 15 Stochastic Gradient Descent Algorithm for Soft-SVM

Parameter: $T \geq 1$
 $\boldsymbol{\theta}^{(1)} \leftarrow \mathbf{0}$ ▷ Or choose random values
for $t \leftarrow 1$ to T **do**
 $\eta^{(t)} \leftarrow \frac{1}{t \cdot \lambda}$ ▷ The learning rate decreases as iterations increase
 $\mathbf{w}^{(t)} \leftarrow \eta^{(t)} \boldsymbol{\theta}^{(t)}$
 pick i uniformly at random from $\{1, \dots, m\}$
 if $y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle < 1$ **then**
 $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)} + y_i \mathbf{x}_i$ ▷ Update the vector
 else
 $\boldsymbol{\theta}^{(t+1)} \leftarrow \boldsymbol{\theta}^{(t)}$
 end if
end for
return $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^T \mathbf{w}^{(t)}$

7.5 Duality (Hard-SVM)

We present Hard-SVM in a different way which is very useful for kernels. In hard-SVM our goal is to find a vector

$$\mathbf{w}_0 = \min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{s.t. } \forall i : y_i \langle \mathbf{w}, \mathbf{x}_i \rangle \geq 1.$$

(considering the homogeneous case and multiplying by $1/2$). Finding the $\mathbf{w}$ that minimizes the function above, is equivalent to find the vector $\boldsymbol{\alpha}$ that solves the *dual problem*:

$$\max_{\boldsymbol{\alpha} \in \mathbb{R}^m: \boldsymbol{\alpha} \geq \mathbf{0}} \left(\sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle \right). \quad (7.7)$$

The solution provides a vector α that defines the support vectors, i.e., the set $\{\mathbf{x}_i : \alpha_i \neq 0\}$. We compute $\mathbf{w}_0 = \sum_{i=1}^m \alpha_i \mathbf{x}_i$. Note that the dual problem requires only to compute inner products $\langle \mathbf{x}_j, \mathbf{x}_i \rangle$, and does not need to consider $\mathbf{x}_i$ by itself; this is useful for non-linear models (kernel tricks).

7.6 Kernel Trick for SVM

So far we have seen SVM only for linear models, but as we know, linear models cannot always be used directly. What we can do is a similar thing that we have seen in subsection 3.3.2:

- We apply a non-linear transformation $\psi(\cdot)$ to each point in training set S and we get $S' = \{(\psi(\mathbf{x}_1), y_1), \dots, (\psi(\mathbf{x}_m), y_m)\}$.
- Then we learn a linear predictor $\hat{h}$ in the transformed space using S' .
- We make predictions for a new instance $\mathbf{x}$ as $\hat{h}(\psi(\mathbf{x}))$.

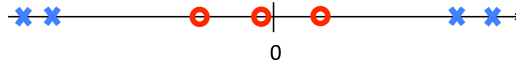


Figure 7.6: Non linearly separable data. $\mathbf{x} = [x]$.

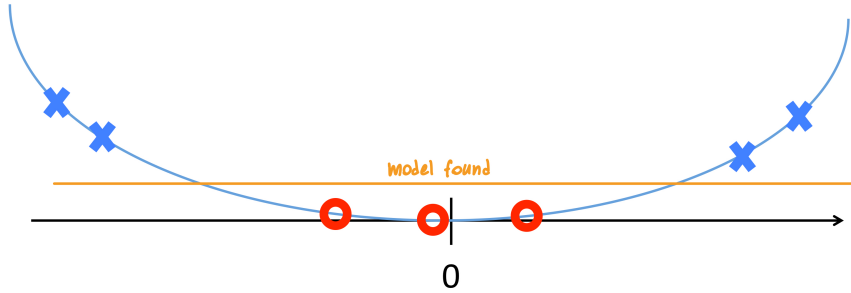


Figure 7.7: Linearly separable data. From $\mathbf{x} = [x]$ to $\mathbf{x}' = [x_1, x_2] = [x, x^2]$.

In our case, we want to learn a model $\hat{h}$ with SVM using the transformed dataset S' . In order to find such model we consider the dual formulation of SVM (7.7), and, as we can see, we only need to be able to compute the dot product $\langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$ for some $\mathbf{x}, \mathbf{x}'$, the dot product of the transformed vectors. In other words we only need to compute the kernel function.

Definition 23 (Kernel function). A kernel function is a function of type:

$$K_{\psi}(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$$

where $\psi(\mathbf{x})$ is a transformation of $\mathbf{x}$.

From the definition above, it seems that, in order to compute the value of the kernel K_ψ , we need to compute also the value of $\psi(\mathbf{x})$. But this isn't always true, this depends on the function $\psi(\mathbf{x})$ that we choose.

Consider for example: $\mathbf{x} = [x_1, \dots, x_d] \in \mathbb{R}^d$. Consider the following transformation (add all possible products to $\mathbf{x}$):

$$\psi(\mathbf{x}) = (1, x_1, x_2, \dots, x_d, x_1x_1, x_1x_2, x_1x_3, \dots, x_dx_d)^T \in \mathbb{R}^{1+d+d^2}$$

Now compute the kernel function:

$$K_\psi(\mathbf{x}, \mathbf{x}') \stackrel{\text{def}}{=} \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle = 1 + \sum_{i=1}^d x_i x'_i + \sum_{i=1}^d \sum_{j=1}^d x_i x_j x'_i x'_j$$

Note that

$$\sum_{i=1}^d \sum_{j=1}^d x_i x_j x'_i x'_j = \left(\sum_{i=1}^d x_i x'_i \right) \left(\sum_{j=1}^d x_j x'_j \right) = (\langle \mathbf{x}, \mathbf{x}' \rangle)^2$$

therefore

$$K_\psi(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle = 1 + \langle \mathbf{x}, \mathbf{x}' \rangle + (\langle \mathbf{x}, \mathbf{x}' \rangle)^2.$$

As we can see, we do not need to compute explicitly the transformation in order to compute the value of the kernel function.

Observation 1. Note also that this is much more efficient: if we compute the value of the kernel function by computing explicitly the transformation, the time complexity is $\Theta(d^2)$; using the implicit computation of the transformation, we get the value of the kernel function by computing a single dot product, so it's $\Theta(d)$.

7.6.1 Commonly used Kernels

List of commonly used kernels:

1. Linear kernel: $\psi(\mathbf{x}) = \mathbf{x}$ (aka the standard SVM). So $K_\psi(\mathbf{x}, \mathbf{x}') = \langle \mathbf{x}, \mathbf{x}' \rangle$.
2. Sigmoid: $K_\psi(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \langle \mathbf{x}, \mathbf{x}' \rangle + \xi)$, for $\gamma, \xi > 0$.
3. Degree- Q polynomial kernel.
4. Gaussian-radial basis function (RBF) kernel.

Note that we do not even care about the definition of the transformation function.

Degree-Q polynomial Kernel For given constants $\gamma > 0$, $\xi > 0$ and for $Q \in \mathbb{N}$, the degree-Q polynomial kernel is

$$K(\mathbf{x}, \mathbf{x}') = (\xi + \gamma \langle \mathbf{x}, \mathbf{x}' \rangle)^Q.$$

The time complexity required is $\Theta(d + \log Q)$.

Gaussian-RBF Kernel For a given constant $\gamma > 0$ the Gaussian-RBF kernel is

$$K(\mathbf{x}, \mathbf{x}') = e^{-\gamma \|\mathbf{x} - \mathbf{x}'\|^2}.$$

If you fix the value $\gamma = 1$ then you can find the definition of the transformation used in this case. If you do that, you will see that $\psi(\mathbf{x})$ has infinite number of dimensions, so it would be impossible to compute the transformation!

7.6.2 Choice of Kernel

In practice: the polynomial kernel is usually used with $Q \leq 10$ and the Gaussian-RBF with $\gamma \in (0, 1]$. Many other choices are possible.

7.7 SVM for Regression

SVMs can be also used for regression. The function to be minimized will be the following:

$$\min_{\mathbf{w}, b} \left(\frac{\lambda}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m V_{\varepsilon}(y_i - \underbrace{(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)}_{\text{predicted value}}) \right)$$

where V_{ε} is the loss function and it is defined as follows:

$$V_{\varepsilon}(r) = \begin{cases} 0 & |r| < \varepsilon \\ |r| - \varepsilon & \text{otherwise.} \end{cases}$$

We can prove that the solution has the form:

$$\mathbf{w} = \sum_{i=1}^m (\alpha_i^* - \alpha_i) \mathbf{x}_i$$

so we can compute the dual of the problem in order to identify the values of the $(\alpha_i^* - \alpha_i)$ that identify the vector of weights $\mathbf{w}$. The final model produced in output is

$$h(\mathbf{x}) = \langle \mathbf{w}, \mathbf{x} \rangle + b = \sum_{i=1}^m (\alpha_i^* - \alpha_i) \langle \mathbf{x}_i, \mathbf{x} \rangle + b$$

where $\alpha^*, \alpha \geq 0$ are the solution of a suitable quadratic programming problem.

Definition 24. *The support vector are the vector $\mathbf{x}_i$ such that $\alpha_i^* - \alpha_i \neq 0$.*

Kernel tricks are also used in a similar way to SVM for classification.

Chapter 8

Neural Networks

8.1 Neuron and activation function

Definition 25 (Neuron and activation function). *A neuron is defined as a function $\mathbf{x} \rightarrow \sigma(\langle \mathbf{v}, \mathbf{x} \rangle)$ with $\mathbf{x} \in \mathbb{R}^d$. $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is called activation function.*

We will consider the following activation functions:

- sign function: $\sigma(a) = \text{sign}(a)$
- threshold function: $\sigma(a) = 1[a > 0]$
- simoid function: $\sigma(a) = \frac{1}{1+e^{-a}}$

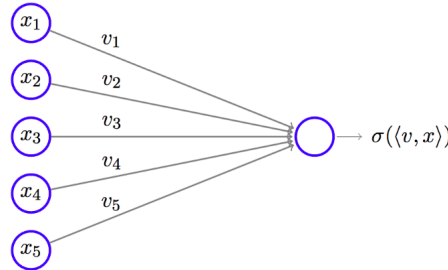


Figure 8.1: Example of neuron with $\mathbf{x} \in \mathbb{R}^5$. Note that if $\sigma(a) = a$ (identity function), then we get as an output of the neuron a linear model. The label v on each arcs are called weights.

8.2 Neural Networks: feedforward neural networks

Neural networks are obtained by connecting many neurons together. We focus on **feedforward neural networks**, defined by a directed acyclic graph $G = (V, E)$ organized in layers.

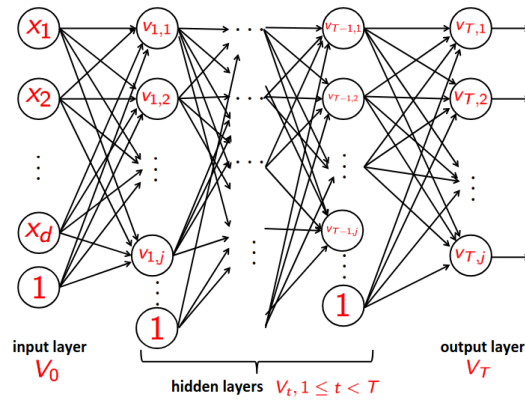


Figure 8.2: The node with the 1 is used to add a bias. The hidden layers are "hidden" because we do not observe them directly. In binary classification the output layer has only one node.

Each edge e has a weight $w(e)$ specified by $w : E \rightarrow \mathbb{R}$. Note that, if we use as activation function $\sigma(a) = a$ for each neuron, apart from the very last one where we use $\sigma(a) = \text{sign}(a)$, we get a linear model as output.

8.3 Notation

8.3.1 Point of View of one node

Consider the node $v_{t+1,j}$, for $0 \leq t < T$. $t+1$ is the layer number, j is the position inside the layer (see figure 8.2). We define:

- $a_{t+1,j}(\mathbf{x})$ as its input when $\mathbf{x}$ is fed to the neural network (the input of the neuron depends on the input of the neural network).
- $o_{t+1,j}(\mathbf{x})$ as its output when $\mathbf{x}$ is fed to the neural network.

Using the definitions above we can compute the input of the node $v_{t+1,j}$ as

$$a_{t+1,j} = \sum_{\substack{r : (v_{t,r}, v_{t+1,j}) \in E \\ \text{Entering arcs}}} \underbrace{w((v_{t,r}, v_{t+1,j}))}_{\text{Weight of the edge}} \underbrace{o_{t,r}(\mathbf{x})}_{\text{Output of node in prev layer}}$$

$$o_{t+1,j}(\mathbf{x}) = \sigma(a_{t+1,j}(\mathbf{x})).$$

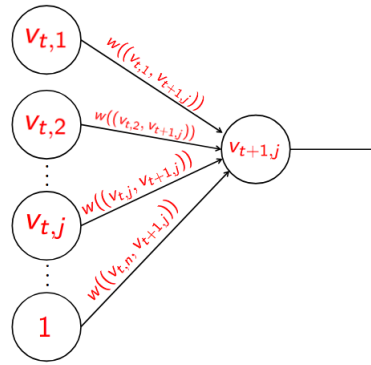


Figure 8.3

8.3.2 Neural network: formalism

Neural network: described by directed acyclic graph $G = (V, E)$ and weight function $w : E \rightarrow \mathbb{R}$

- $V = \cup_{t=0}^T V_t$, $V_i \cap V_j = \emptyset \quad \forall i \neq j$
- $e \in E$ can only go from V_t to V_{t+1} for some t
- $V_0 =$ input layer
- $V_T =$ output layer
- $V_t, 0 < t < T =$ hidden layers
- $T =$ depth
- $|V| =$ size of the network
- $\max_t |V_t| =$ width of the network

Notes:

- for binary classification and regression (1 variable): output layer has 1 node;
- different layers could have different activation functions (e.g., output layer).

8.4 Hypothesis Set of a Neural Network

First of all we need to define the **architecture of the neural network**, i.e., we need to specify (V, E, σ) . Once we have specified the architecture and the function $w : E \rightarrow \mathbb{R}$, we obtain a function:

$$h_{V,E,\sigma,w} : \mathbb{R}^{|V_0|-1} \rightarrow \mathbb{R}^{|V_T|}$$

(the -1 in the domain is for the bias). Any set of such functions can serve as a hypothesis class for learning. We denote the hypothesis class by:

$$H_{V,E,\sigma} = \{h_{V,E,\sigma,w} : w \text{ is a mapping from } E \text{ to } \mathbb{R}\}.$$

8.5 Expressive power of neural network

Using a neural network, what type of functions can be implemented?

Proposition 6. *For every d , there exists a graph (V, E) of depth 2 such that $H_{V,E,\text{sign}}$ contains all the functions from $\{-1, 1\}^d$ to $\{-1, 1\}$.*

The proposition above states that we need a neural network with one hidden layer, in order to learn any boolean function. How can we construct such neural network? There exists a general procedure:

1. Let $f : \{-1, 1\}^d \rightarrow \{-1, 1\}$ be any boolean function.
2. The goal is to build a neural network that computes f , so if the input is $\mathbf{x}$, then the prediction must be $f(\mathbf{x})$.
3. Consider all the vectors $\mathbf{x}$ such that $f(\mathbf{x}) = 1$.
4. For each such vector $\mathbf{x}$:
 - (a) Add a neuron in the only hidden layer that "corresponds" to $\mathbf{x}$.
 - (b) This neuron computes the following function:

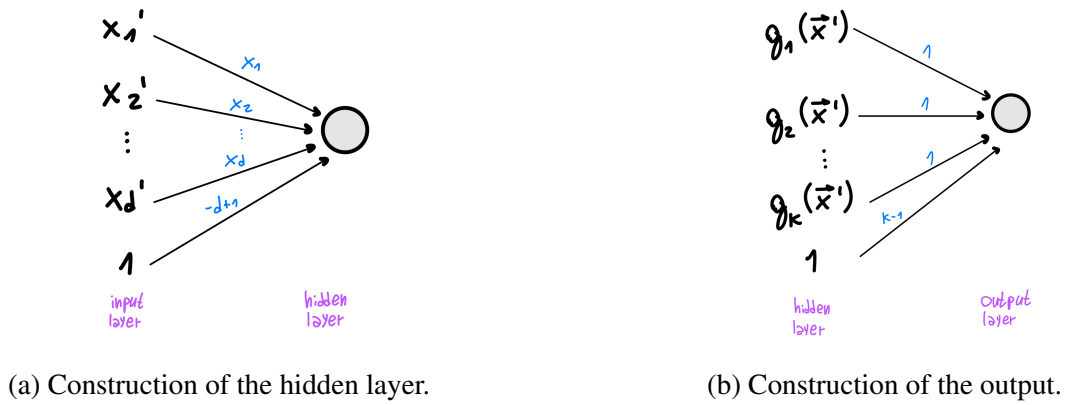
$$g(\mathbf{x}') = \text{sign}(\langle \mathbf{x}, \mathbf{x}' \rangle - d + 1)$$

where $\mathbf{x}$ are the weights on the edges from the input layer to the neuron; $-d + 1$ is the weight on the edge from the constant to the neuron.

5. The output node computes the following function:

$$h(\mathbf{x}') = \text{sign} \left(\sum_{i=1}^k g_i(\mathbf{x}') + k - 1 \right)$$

where k is the number nodes in the only hidden layer; $k - 1$ is the weight on the edge from the constant to the neuron. Taking the sum of all the $g_i(\mathbf{x}')$ (the output of the neuron i from the hidden layer) means that the weights on the edges from each node of the hidden layer to the output node are all set to 1.

Figure 8.4: Neural network that implements the boolean function f .

Consider an example. Let $\mathbf{x} = [x_1, x_2] \in \{-1, 1\}^2$ and let the training data be represented by the following table:

x_1	x_2	y
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

Figure 8.5: XOR

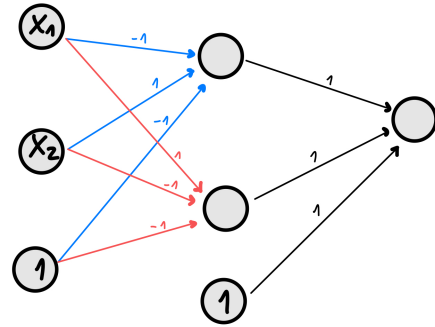


Figure 8.6

The training set represents the XOR function, that is not linear. Such dataset cannot be perfectly classified with a linear models, but the neural networks can perfectly classify it, therefore NNs are richer models than linear models.

Proposition 7. For every d , let $s(d)$ be the minimal integer such that there exists a graph (V, E) with $|V| = s(d)$ such that $H_{V, E, \text{sign}}$ contains all functions from $\{-1, 1\}^d$ to $\{-1, 1\}$. Then $s(d)$ is an exponential function of d .

Neural networks can implement any boolean function, the downside is that that the number of neurons required grows exponentially. This does not depend on the activation function, we can use the sigmoid function instead of the sign function, the result is the same (the construction of the NN is different though).

What about regression tasks?

Proposition 8. *For every fixed $\varepsilon > 0$ and every Lipschitz function¹ $f : [-1, 1]^d \rightarrow [-1, 1]$ it is possible to construct a neural network such that for every input $\mathbf{x} \in [-1, 1]^d$ (we have to normalize the input) the output of the neural network is in $[f(\mathbf{x}) - \varepsilon, f(\mathbf{x}) + \varepsilon]$*

Also for regression tasks we can construct a neural network that almost predicts exactly the function $f(\mathbf{x})$, for any function f . Neural networks are **universal approximators**.

But again the downside is that the size of the neural network grows exponentially in d .

Proposition 9. *Fix some $\varepsilon \in [0, 1]$. For every d , let $s(d)$ be the minimal integer such that there exists a graph (V, E) with $|V| = s(d)$ such that $H_{V, E, \sigma}$, with $\sigma = \text{sigmoid}$, can approximate, with precision ε , every 1-Lipschitz function $f : [-1, 1]^d \rightarrow [-1, 1]$. Then $s(d)$ is exponential in d .*

8.6 Runtime of Learning Neural Networks

Applying the Error Risk Minimization rule with respect to $H_{V, E, \text{sign}}$ is computationally difficult (NP-hard) even for small neural networks.

Proposition 10. *Let $k \geq 3$. For every d , let (V, E) be a layered graph with d input nodes, $k + 1$ nodes at the (only) hidden layer, where one of them is the constant neuron, and a single output node. Then, it is NP-hard to implement the ERM rule with respect to $H_{V, E, \text{sign}}$.*

Even if we want to find an hypothesis h close to the ERM or we consider other activation functions (e.g sigmoid) or if we consider a larger neural network (more deep), the problem remains computationally infeasible.

So how do we train neural networks? We can use heuristics: a widely used heuristic for training neural networks relies on the SGD framework. In a previous chapter we have studied the SDG algorithm, but we were considering only convex loss functions. In neural networks, the loss function is highly nonconvex, nevertheless we can use SDG algorithm and hope it will find a good solution: in many practical cases works good.

¹From wikipedia: "a Lipschitz continuous function is limited in how fast it can change: there exists a real number such that, for every pair of points on the graph of this function, the absolute value of the slope of the line connecting them is not greater than this real number".

8.7 SGD for Neural Networks

8.7.1 Notation

Consider layer t , $0 < t < T$ (not the input layer nor the output layer). Let:

$d^{(t)} :=$ the number of nodes in layer t .

Each hidden layer has a constant node 1 and the values of nodes for variables $v_{t,1} \dots, v_{t,d^{(t)}}$.

Define:

$w_{ij}^{(t)} :=$ weight of the arc from node $v_{t-1,i}$ to $v_{t,j}$

$$\mathbf{v}^{(t)} := \begin{pmatrix} 1 \\ v_{t,1} \\ \vdots \\ v_{t,d^{(t)}} \end{pmatrix} := \text{outputs of layer } t$$

$$\mathbf{w}_j^{(t)} := \begin{pmatrix} w_{0j}^{(t)} \\ w_{1j}^{(t)} \\ \vdots \\ w_{d^{(t-1)}j}^{(t)} \end{pmatrix} := \text{weights on arcs entering in node } j \text{ in layer } t$$

then

$$v_{t,j} := \sigma \left(\underbrace{\langle \mathbf{w}_j^{(t)}, \mathbf{v}^{(t-1)} \rangle}_{\text{Input of the node}} \right) := \text{output of the node } j \text{ in layer } t.$$

We can rewrite the vector $\mathbf{v}^{(t)}$ as follows:

$$\mathbf{v}^{(t)} := \begin{bmatrix} 1 \\ v_{t,1} \\ \vdots \\ v_{t,d^{(t)}} \end{bmatrix} = \begin{bmatrix} 1 \\ \sigma \left(\langle \mathbf{w}_1^{(t)}, \mathbf{v}^{(t-1)} \rangle \right) \\ \vdots \\ \sigma \left(\langle \mathbf{w}_{d^{(t)}}^{(t)}, \mathbf{v}^{(t-1)} \rangle \right) \end{bmatrix}$$

Define:

$a_{t,j} := \langle \mathbf{w}_j^{(t)}, \mathbf{v}^{(t-1)} \rangle :=$ input of node j in layer t

$$\mathbf{a}^{(t)} := \begin{bmatrix} a_{t,1} \\ \vdots \\ a_{t,d^{(t)}} \end{bmatrix}, \quad \sigma(\mathbf{a}^{(t)}) := \begin{bmatrix} \sigma(a_{t,1}) \\ \vdots \\ \sigma(a_{t,d^{(t)}}) \end{bmatrix}$$

In the end we can rewrite the vector $\mathbf{v}^{(t)}$ as follows:

$$\mathbf{v}^{(t)} = \begin{bmatrix} 1 \\ \sigma(\mathbf{a}^{(t)}) \end{bmatrix}$$

Define:

$$\mathbf{w}^{(t)} := [\mathbf{w}_1^{(t)}, \dots, \mathbf{w}_j^{(t)}, \dots, \mathbf{w}_{d(t)}^{(t)}] := \text{matrix of weights of edges from layer } t-1 \text{ to } t.$$

Then we can redefine the input vector $\mathbf{a}^{(t)}$ as follows:

$$\mathbf{a}^{(t)} := (\mathbf{w}^{(t)})^T \mathbf{v}^{(t-1)}$$

8.7.2 Forward Propagation Algorithm

Algorithm 16 Neural Network Forward Pass

```

1: Input:  $\mathbf{x} = (x_1, \dots, x_d)^T$ ; Neural Network (NN) with 1 output node
2: Output: Prediction  $y$  of the NN
3:  $\mathbf{v}^{(0)} \leftarrow (1, x_1, \dots, x_d)^T$  ▷ Input of the NN
4: for  $t \leftarrow 1$  to  $T$  do
5:    $\mathbf{a}^{(t)} \leftarrow (\mathbf{w}^{(t)})^T \mathbf{v}^{(t-1)}$  ▷ Input of the layer  $t$ 
6:    $\mathbf{v}^{(t)} \leftarrow (1, \sigma(\mathbf{a}^{(t)})^T)^T$  ▷ Output of layer  $t$ 
7: end for
8:  $y \leftarrow \sigma(\mathbf{a}^{(T)})$  ▷ Output of the NN
9: return  $y$ 

```

Using the algorithm above, we can compute the prediction. In the next subsection we will see how to compute the values of the weights.

8.7.3 Learning NN parameters

How we compute the value of the weights $w_{ij}^{(t)}$? Given a training data S we need to pick the weights for every i, j, t (defining a specific model h) minimizing the training error $L_S(h) = \frac{1}{m} \sum_{i=1}^m l(h, (\mathbf{x}_i, y_i))$.

We use the Gradient Descent; let $L_S(h)$ be a function of $\mathbf{w}^{(t)}$ for all $1 \leq t \leq T$. The GD use the following update rule:

$$\mathbf{w}^{(t)} \leftarrow \mathbf{w}^{(t)} - \eta \nabla L_S(\mathbf{w}^{(t)}) \tag{8.1}$$

where $\nabla L_S(\mathbf{w}^{(t)})$ is the gradient of L_S and η is the learning parameter. How do we compute the gradient?

$$\frac{\partial L_S}{\partial \mathbf{w}^{(t)}} = \frac{\partial}{\partial \mathbf{w}^{(t)}} \left(\frac{1}{m} \sum_{i=1}^m l(h, (\mathbf{x}_i, y_i)) \right) = \frac{1}{m} \sum_{i=1}^m \frac{\partial l(h, (\mathbf{x}_i, y_i))}{\partial \mathbf{w}^{(t)}}$$

so we need to compute $\frac{\partial l}{\partial \mathbf{w}^{(t)}}$ for every t such that $1 \leq t \leq T$.

Definition 26 (Sensitivity vector for layer t). *The sensitivity vector for layer t is*

$$\delta^{(t)} = \frac{\partial l}{\partial \mathbf{a}^{(t)}} = \begin{bmatrix} \frac{\partial l}{\partial a_{t,1}} \\ \vdots \\ \frac{\partial l}{\partial a_{t,d^{(t)}}} \end{bmatrix} = \begin{bmatrix} \delta_1^{(t)} \\ \vdots \\ \delta_{d^{(t)}}^{(t)} \end{bmatrix}$$

The vector $\delta^{(t)}$ quantifies how the training error changes with the inputs to the t layer $\mathbf{a}^{(t)}$ (before the nonlinear transformation).

Consider a single weight $w_{i,j}^{(t)}$: a change in his value changes only $a_{t,j}$ and therefore by chain rule we can write (omitting the steps):

$$\frac{\partial l}{\partial w_{ij}^{(t)}} = \frac{\partial l}{\partial a_{t,j}} \cdot \frac{\partial a_{t,j}}{\partial w_{ij}^{(t)}} = \delta_j^{(t)} \cdot v_{t-1,i} \quad (8.2)$$

(where $v_{t-1,i}$ is the output of node i in layer $t-1$).

Therefore, to compute the gradient of L_S , we only need to compute the value of the sensitive vector $\delta^{(t)} = \frac{\partial l}{\partial \mathbf{a}^{(t)}}$ for every layer t ; the value of $v_{t-1,i}$ can be **computed by the forward propagation algorithm**.

Consider the definition of the sensitive vector $\delta^{(t)}$ and consider one component: since the loss function l depends from $a_{t,j}$ only trough $v_{t,j}$, then using the chain rule we have:

$$\begin{aligned} \delta_j^{(t)} &= \frac{\partial l}{\partial a_{t,j}} \\ &= \frac{\partial l}{\partial v_{t,j}} \cdot \frac{\partial v_{t,j}}{\partial a_{t,j}} \\ &= \frac{\partial l}{\partial v_{t,j}} \cdot \sigma'(a_{t,j}) \quad (\text{using the definition of } v_{t,j}) \end{aligned}$$

Last step: consider $\frac{\partial l}{\partial v_{t,j}}$, we need to understand how loss l changes due to changes in $v_{t,j}$:

- Change in $\mathbf{v}^{(t)}$ affects only $\mathbf{a}^{(t+1)}$ (i.e. the changes in output layer of t affects the input of

the next layer $t + 1$) and then the loss function l ,

- So the changes in $v_{t,j}$ can affect every $a_{t+1,k}$ for all $k = 1, \dots, d^{(t+1)}$ (i.e. the change in the output of node j in layer t affects the input of each node in the next layer $t + 1$).

We can sum chain rule contributions:

$$\begin{aligned} \frac{\partial l}{\partial v_{t,j}} &= \sum_{k=1}^{d^{(t+1)}} \frac{\partial a_{t+1,k}}{\partial v_{t,j}} \cdot \frac{\partial l}{\partial a_{t+1,k}} \\ &= \sum_{k=1}^{d^{(t+1)}} w_{jk}^{(t+1)} \cdot \delta_k^{(t+1)} \end{aligned}$$

To summarize: we can compute each entry of the sensitive vector $\delta^{(t)}$ for layer t as follows:

$$\delta_j^{(t)} = \sigma'(a_{t,j}) \cdot \sum_{k=1}^{d^{(t+1)}} w_{jk}^{(t+1)} \cdot \delta_k^{(t+1)} \quad (8.3)$$

Note that:

- $\sigma'(a_{t,j})$ depends on the activation function σ chosen.
- In order to compute $\delta_j^{(t)}$ (i.e. the j -th component of the sensitive vector of layer t) we need to compute $\delta^{(t+1)}$ (i.e. the sensitive vector of the next layer). **Back propagation.**
- The back propagation starts from the output layer. We need to compute $\delta^{(L)} = \frac{\partial l}{\partial \mathbf{a}^{(L)}}$ (i.e. the sensitivity vector of the final layer), that depends on the loss function l chosen.

8.7.4 Compute sensitivities vectors

The following algorithm computes the value of the sensitivities vectors $\delta^{(t)}$ for each layer t , for a given data point $(\mathbf{x}_i, y_i)$, using the equation 8.3.

Algorithm 17

- 1: **Input:** Data point $(\mathbf{x}_i, y_i)$, Neural Network (NN) with weights $w_{ij}^{(t)}$ for $1 \leq t \leq T$
 - 2: **Output:** $\delta^{(t)}$ for $t = 1, \dots, T$
 - 3: Compute $\mathbf{a}^{(t)}$ and $\mathbf{v}^{(t)}$ for $t = 1, \dots, T$ ▷ Using forward propagation
 - 4: $\delta^{(T)} \leftarrow \frac{\partial \ell}{\partial \mathbf{a}^{(T)}}$ ▷ Use the definition for the output layer.
 - 5: **for** $t \leftarrow T - 1$ **downto** 1 **do** ▷ Backward
 - 6: $\delta_j^{(t)} \leftarrow \sigma'(a_{t,j}) \cdot \sum_{k=1}^{d^{(t+1)}} w_{jk}^{(t+1)} \delta_k^{(t+1)} \quad \forall j = 1, \dots, d^{(t)}$
 - 7: **end for**
 - 8: **return** $\delta^{(1)}, \dots, \delta^{(T)}$
-

8.7.5 Backpropagation Algorithm

Backpropagation algorithm, based on SGD², is used to train a neural network.

Algorithm 18 Training Neural Networks with Backpropagation

```

1: Input: Training data  $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$ , NN (no weights  $w_{ij}^{(t)}$ )
2: Output: NN with weights  $w_{ij}^{(t)}$ 
3: Initialize  $w_{ij}^{(t)}$  for all  $i, j, t$ 
4: for  $s = 0, 1, 2, \dots$  do ▷ Until convergence
5:   Pick  $(\mathbf{x}_k, y_k)$  at random from training data
6:   Compute  $v_{t,j}$  for all  $j, t$  from  $(\mathbf{x}_k, y_k)$  ▷ Forward propagation
7:   Compute  $\delta_j^{(t)}$  for all  $j, t$  from  $(\mathbf{x}_k, y_k)$  ▷ Backward propagation
8:   Update weights:  $w_{ij}^{(t)} \leftarrow w_{ij}^{(t)} - \eta v_{t-1,i} \cdot \delta_j^{(t)}$  for all  $i, j, t$  ▷ SGD update
9:   if converged then
10:     return  $w_{ij}^{(t)}$  for all  $i, j, t$ 
11:   end if
12: end for

```

Notes on the algorithm:

- Preprocessing: all the inputs are normalized and centered.
- Initialization of the weights of the NN: random values around 0, for instance $w_{ij}^{(t)} \sim U(-0.7, 0.7)$ (uniform distribution) or $w_{ij}^{(t)} \sim N(0, \sigma^2)$ with variance. If we set all the weights to zero, then all neurons get the same weights (by construction).
- When to stop: usually a combination of small training error; small marginal improvement in error; upper bound on number of iteration.
- Local minima: usually $L_S(h)$ has multiple local minima, therefore is useful to run the SGD multiple times with random initial weights.

8.8 Regularized neural networks: squared weight decay regularizer

As we know, Regularized Loss Minimization it is used in order to prevent overfitting and to find simpler model.

²Means that the update of the weights is computed using $\nabla l(h, (\mathbf{x}_i, y_i))$ for a randomly chosen point $(\mathbf{x}_i, y_i)$: is more efficient that computing the gradient of L_S .

Regularized Loss Minimization for NN: find an hypothesis h that minimizes

$$\arg \min_{h \in H_{V,E,\sigma}} \left\{ L_S(h) + \frac{\lambda}{2} \sum_{i,t,j} (w_{ij}^{(t)})^2 \right\} \quad (8.4)$$

where λ is the regularization parameter. We can find the hypothesis h using the SGD algorithm, in this case the gradient for layer t is $\nabla(L_S(h)) + \lambda \mathbf{w}^{(t)}$.

Note that other regularization are possible.

8.9 Geometric Intuition

Geometric illustrations of functions $f: \mathbb{R}^2 \rightarrow \{-1, 1\}$ expressed using a neural network with the *sign* activation function.

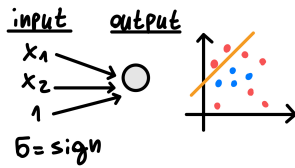


Figure 8.7: One single neuron implements an halfspace predictor.

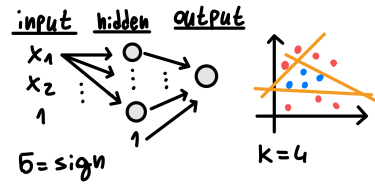


Figure 8.8: k nodes in the only hidden layer: $k - 1$ neurons implement an halfspace predictor. The single output neuron applies an halfspace on top of the binary outputs of the hidden layer.

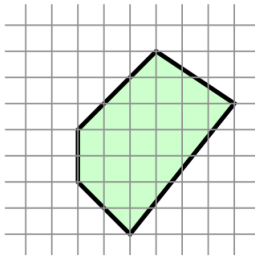


Figure 8.9: k neurons in the hidden layer can express all convex polytopes with $k - 1$ faces.

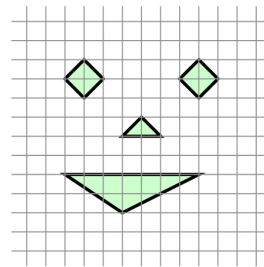


Figure 8.10: With two hidden layers we get a network that computes the union of polytopes.

Chapter 9

Deep Learning

In theory we can use neural networks with only one hidden layer to implement any function, but we have seen in the previous chapter that the size of the NN grows exponentially. Deep neural networks have the same power of NN but in practice they require much less neurons in order to implement complex functions.

Problems with the traditional NNs:

- Require huge number of edges $\Rightarrow$ huge number of weights.
- Domain structure is not taken into account: for example if the input is an image, the near by pixels are related, so it is useful in the neural networks to have similar/related neurons.

9.1 Convolutional Neural Networks (CNNs)

CNNs are a specialized kind of neural network for processing data that has a known grid-like topology (also work with graphs).

Definition 27. *CNNs are neural networks that use convolution in place of general matrix multiplication in at least one of their layers.*

Using convolution:

- **Leads to sparse interaction:** some edges "don't appear", i.e., the weights are fixed to 0.
- **Parameter sharing:** some neurons share the same weights $\Rightarrow$ we learn less parameters.

Those two properties reduce the number of parameters of the neural network, so we get simpler model.

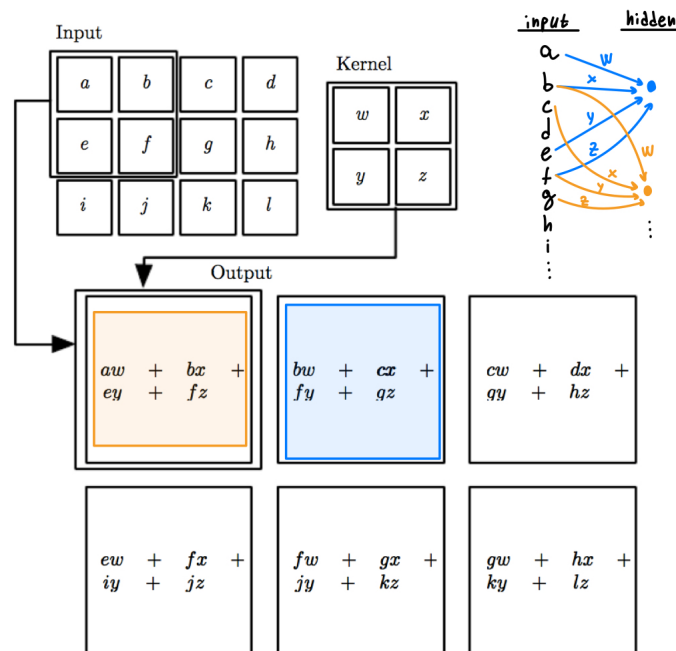


Figure 9.1: Discrete convolution. The kernel K is learned from the training data. Note that many edges do not exist: sparse interaction.

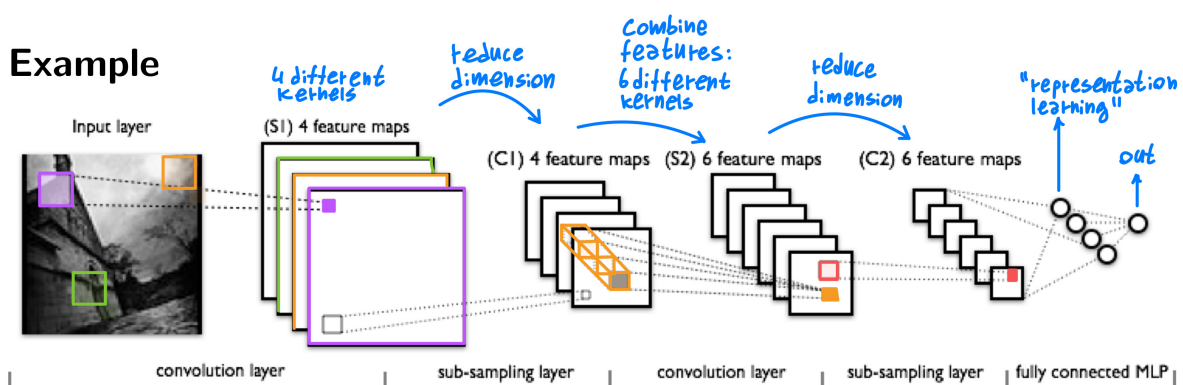


Figure 9.2: Example of CNN.

In a CNN there are **convolutional layers** (implement convolution) and other type of layers. The convolutional layers models consists of several filters/kernels. After a convolutional layer we can find a non linear function (usually *ReLU* because is easy to compute and leads to sparse activation) or a pooling layer, often combined with subsampling.

Pooling functions replaces the output of a network at a certain location with a summary statistic of the nearby outputs. Popular functions are: *max pooling*, *average pooling*, weighted average based on the distance from the network location, etc.

Subsampling is performed by setting the *stride* to a value > 0 .

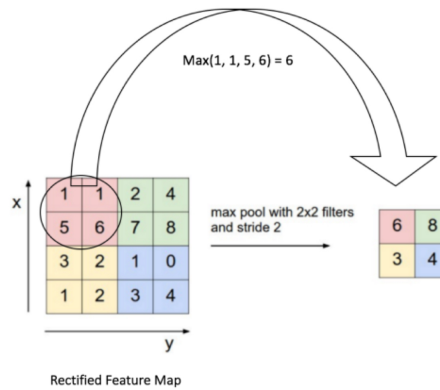


Figure 9.3: Stride and max pooling.

Pooling and subsampling is very useful for reducing the number of parameters (prevents overfitting) and also reduce the computation.

Chapter 10

Learning via Uniform Convergence

10.1 Uniform Convergence and Learnability

The Error Risk Minimization works as follows: upon giving a training set S , the learner evaluates the risk of each $h \in H$ on the given sample and outputs a member of H that minimizes this empirical risk. The hope is that an h that minimizes the empirical risk has also the minimum generalization error in our class H . For that, it suffices to ensure that the empirical risk of all members in H are good approximations of their generalization error.

Definition 28 (ϵ -representative). *A training set S is called ϵ -representative (with respect to domain Z , hypothesis class H , loss function l and distribution D) if*

$$\forall h \in H, \quad |L_S(h) - L_D(h)| \leq \epsilon.$$

That is, we say that S is ϵ -representative if the generalization error of each $h \in H$ is close to the training error of h :

$$\forall h \in H, \quad L_S(h) - \epsilon \leq L_D(h) \leq L_S(h) + \epsilon.$$

Proposition 11. *Assume that the training set S is $\epsilon/2$ -representative (with respect to domain Z , hypothesis class H , loss function l and distribution D). Then, any output of $ERM_H(S)$ (i.e. any $h_S \in \arg \min_{h \in H} L_S(h)$) satisfies*

$$L_D(h_S) \leq \min_{h \in H} L_D(h) + \epsilon.$$

So, if the training set S is $\epsilon/2$ -representative, then the ERM learning paradigm is guaranteed to return a good hypothesis.

Proof. For every $h \in H$ we have:

$$\begin{aligned}
 L_D(h_S) &\leq L_S(h_S) + \frac{\varepsilon}{2} && S \text{ is } \varepsilon/2\text{-representative} \\
 &\leq L_S(h) + \frac{\varepsilon}{2} && h_S \in \arg \min_{h \in H} L_S(h) \\
 &\leq L_D(h) + \frac{\varepsilon}{2} + \frac{\varepsilon}{2} && S \text{ is } \varepsilon/2\text{-representative} \Rightarrow L_S(h) \leq L_D(h) + \frac{\varepsilon}{2}
 \end{aligned}$$

Therefore:

$$\Rightarrow L_D(h_S) \leq L_D(h) + \varepsilon$$

and this is true for every $h \in H \Rightarrow$ is true for $h = \arg \min_{h' \in H} L_D(h') \Rightarrow L_D(h_S) \leq \arg \min_{h' \in H} L_D(h') + \varepsilon. \quad \square$

The previous proposition implies that: ERM rule is an agnostic PAC learner (recall the definition 21) if, with probability of at least $1 - \delta$ over the random choice of a training set S , the training set S is an ε -representative set.

The Uniform Convergence condition formalize this requirement.

Definition 29 (Uniform Convergence). *A hypothesis class H has the uniform convergence property (with respect to domain Z and a loss function l) if there exist a function $m_H^{UC} : (0, 1)^2 \rightarrow \mathbb{N}$ such that for every $\varepsilon, \delta \in (0, 1)$ and for every probability distribution D over Z , if S is a sample of $m \geq m_H^{UC}(\varepsilon, \delta)$ iid examples drawn from D , then with probability $\geq 1 - \delta$, S is ε -representative.*

The function m_H^{UC} measure the minimal sample complexity of obtaining the uniform convergence property, that is, it indicates the minimum number of samples that we need in order to ensure that the training set S is ε -representative with probability $1 - \delta$.

The following proposition follows directly from the definition of uniform convergence and the previous proposition.

Proposition 12. *If a class H has the uniform convergence property with a function m_H^{UC} then the class is agnostically PAC learnable with the sample complexity $m_H(\varepsilon, \delta) \leq m_H^{UC}(\varepsilon/2, \delta)$ (at most we need $m_H^{UC}(\varepsilon/2, \delta)$ samples, maybe less than that). Furthermore, in that case the ERM_H paradigm is a successful PAC learner for H .*

The question now is: what classes of hypothesis have uniform convergence?

10.2 Finite Classes are Agnostic PAC Learnable

We want to prove that finite sets of hypotheses are agnostic PAC learnable under some restriction for the loss function.

Proposition 13. *Let H be a finite hypothesis class, let Z be a domain, and let $l : H \times Z \rightarrow [0, 1]$ (restriction) be a loss function. Then:*

- H enjoys the uniform convergence property with sample complexity

$$m_H^{UC}(\epsilon, \delta) \leq \underbrace{\left\lceil \frac{\log(2|H|/\delta)}{2\epsilon^2} \right\rceil}_{:=m}$$

- H is agnostically PAC learnable using the ERM algorithm with sample complexity

$$m_H(\epsilon, \delta) \leq m_H^{UC}(\epsilon/2, \delta) \leq \left\lceil \frac{2\log(2|H|/\delta)}{\epsilon^2} \right\rceil$$

Idea of the proof: we need to prove the first point, i.e., we want to show that every finite hypothesis class H enjoy the uniform convergence property; after that, the second point follows from the proposition 12.

Useful tool for the proof:

Definition 30 (Hoeffding's Inequality). *Let $\theta_1, \dots, \theta_m$ be a sequence of iid random variables and assume that for all i , $E[\theta_i] = \mu$ and $P[a \leq \theta_i \leq b] = 1$. Then, for any $\epsilon > 0$*

$$P \left[\left| \frac{1}{m} \sum_{i=1}^m \theta_i - \mu \right| > \epsilon \right] \leq 2e^{-\frac{2m\epsilon^2}{(b-a)^2}}$$

Where:

- $\frac{1}{m} \sum_{i=1}^m \theta_i$ is the average of the observation ($\sim$ the training error);
- μ is the expectation ($\sim$ the generalization error);
- $\left| \frac{1}{m} \sum_{i=1}^m \theta_i - \mu \right| < \epsilon$ means that the two values are more or less the same, and this holds with a probability $\leq 2e^{-\frac{2m\epsilon^2}{(b-a)^2}}$.

Proof. We want to show that every finite hypothesis class H enjoy the uniform convergence property. In order to do that, we need to fix $(\epsilon, \delta) \in (0, 1)^2$, and find the sample size m such that,

for any distribution D , with probability $1 - \delta$ (over the choice of $S = \{z_1, \dots, z_m\}$, $z_i = (\mathbf{x}_i, y_i)$), we have that

$$\forall h \in H \quad |L_S(h) - L_D(h)| \leq \varepsilon$$

that is, S is ε -representative.

In other words what we want to verify is:

$$D^m(\{S : \forall h \in H \quad |L_S(h) - L_D(h)| \leq \varepsilon\}) \geq 1 - \delta$$

where $S = \{z_1, \dots, z_m\}$, $z_i = (\mathbf{x}_i, y_i)$ are iid from D .

Equivalently, we need to show:

$$(\star) \quad D^m(\{S : \exists h \in H \quad |L_S(h) - L_D(h)| > \varepsilon\}) \leq \delta$$

We have:

$$\{S : \exists h \in H \quad |L_S(h) - L_D(h)| > \varepsilon\} = \bigcup_{h \in H} \{S : |L_S(h) - L_D(h)| > \varepsilon\}$$

From $(\star)$ and using the observation above:

$$(\star\star) \quad D^m(\{S : \exists h \in H \quad |L_S(h) - L_D(h)| > \varepsilon\}) \leq \sum_{h \in H} D^m(\{S : |L_S(h) - L_D(h)| > \varepsilon\})$$

by using the union bound. Now we want to bound each term of the sum in $(\star\star)$.

Recall that: $L_D(h) := E_{z \sim D}[l(h, z)]$ and $L_S(h) := \frac{1}{m} \sum_{i=1}^m l(h, z_i)$.

Important: since each z_i is sampled iid from D (i.e. $z_i \sim D$), the expected value of the random variable $l(h, z_i)$ is $L_D(h)$:

$$E[l(h, z_i)] = E_{z \in D}[l(h, z)] := L_D(h).$$

Therefore it follows that:

$$\begin{aligned}
 E[L_S(h)] &= E \left[\frac{1}{m} \sum_{i=1}^m l(h, z_i) \right] && \text{by def of } L_S(h) \\
 &= \frac{1}{m} \sum_{i=1}^m \underbrace{E[l(h, z_i)]}_{=E_{z \sim D}[l(h, z)] := L_D(h)} && \text{by linearity of } E \\
 &= \frac{1}{m} \sum_{i=1}^m L_D(h) \\
 &= L_D(h)
 \end{aligned}$$

Hence, the quantity $|L_S(h) - L_D(h)|$ is the deviation of the random variable $L_S(h)$ from its expectation.

Let θ_i be the random variable given by $l(h, z_i)$. Since h is fixed and z_i is sampled iid from D , it follows that $\theta_1, \dots, \theta_m$ are iid random variables.

Note that $L_S(h) = \frac{1}{m} \sum_{i=1}^m \theta_i$. Let's define $\mu := L_D(h)$. Given the assumption that $l : H \times Z \rightarrow [0, 1]$ we have that $\theta_i \in [0, 1] \forall i = 1, \dots, m$.

Therefore we can apply the Hoeffding's inequality with $a_i = 0, b_i = 1 \forall i = 1, \dots, m$. We get:

$$D^m(\{S : |L_S(h) - L_D(h)| > \varepsilon\}) = P \left[\left| \frac{1}{m} \sum_{i=1}^m \theta_i - \mu \right| > \varepsilon \right] \leq 2e^{-2m\varepsilon^2}$$

Combining the above with $(\star\star)$:

$$D^m(\{S : \exists h \in H \quad |L_S(h) - L_D(h)| > \varepsilon\}) \leq \sum_{h \in H} 2e^{-2m\varepsilon^2} = 2|H|e^{-2m\varepsilon^2}$$

By choosing:

$$m \geq m_H^{UC}(\varepsilon, \delta) := \log \left(\frac{2|H|}{\delta} \right) \cdot \frac{1}{2\varepsilon^2}$$

we get:

$$D^m(\{S : |L_S(h) - L_D(h)| > \varepsilon\}) \leq 2|H|e^{-2 \log \left(\frac{2|H|}{\delta} \right) \cdot \frac{1}{2\varepsilon^2} \varepsilon^2} = \delta$$

For instance we can choose $m := \left\lceil \log \left(\frac{2|H|}{\delta} \right) \cdot \frac{1}{2\varepsilon^2} \right\rceil$; with this choice, we have proved that the training set S is ε -representative and therefore the hypothesis class H enjoys the uniform convergence property. $\square$

Chapter 11

VC-Dimension

In the previous chapter we have seen that finite classes are learnable. But what about infinite classes of hypothesis? In practice we work with infinite classes of hypothesis, so what we can say about them? Can infinite size classes be learnable, and, if so, what determines their sample complexity?

In the next sections we will focus on binary classification and 0-1 loss function. But similar results apply to other learning tasks and losses.

11.1 Restrictions

Definition 31 (Restriction of H to C). *Let H be a class of functions from X to $\{0, 1\}$ and let $C = \{c_1, \dots, c_m\} \subset X$ (some samples). The restriction H_C of H to C is:*

$$H_C = \{[h(c_1), \dots, h(c_m)] : h \in H\}$$

where we represent each function from C to $\{0, 1\}$ as a vector in $\{0, 1\}^{|C|}$.

H_C is the set of functions $h : C \rightarrow \{0, 1\}$ that can be derived from H .

H can be a class of hypotheses of infinite size. But what about H_C ? Can H_C be infinite? The answer is no! In H_C we have binary vectors with dimension equal to $|C|$. Therefore there exist at most $2^{|C|}$ different vectors. In other words:

$$1 \leq |H_C| \leq 2^{|C|}. \quad (11.1)$$

This means that some hypothesis $h \in H$ are "useless": they do not define new ways of assign the labels to the samples in C . That is, the number of ways of assigning the values of the labels

for the samples in C is a finite number. What we are going to say is that, the real size of H is $|H_C|$, which is finite.

11.2 VC-dimension and Shattering

Definition 32 (Shattering). *A hypothesis class H shatters a finite set $C \subset X$ if the restriction H_C of H to C is the set of all functions from C to $\{0, 1\}$. That is, $|H_C| = 2^{|C|}$.*

In other words by using the hypotheses in H_C we can get all the possible labels.

Definition 33 (VC-dimension). *The VC-dimension $VCdim(H)$ of a hypothesis class H , is the maximal size of a set $C \subset X$ that can be shattered by H . If H can shatter sets of arbitrarily large size, we say that H has infinite VC-dimension.*

We say that the "effective" size of H is $VCdim(H)$.

Note that:

- If H can shatter sets of arbitrarily large size then we say that $VCdim(H) = +\infty$.
- If $|H| < +\infty$ then $VCdim(H) \leq \log_2 |H|$.

The VC-dimension measures the complexity of H : how large a dataset that is perfectly classified using the functions in H can be.

To show that $VCdim(H) = d$ we need to show that: $VCdim(H) \geq d$ and $VCdim(H) \leq d$. In other words we have to prove that it exists a set C of size d which is shattered by H and every set of size $d + 1$ is not shattered by H .

(See some examples in the notes).

11.3 The Fundamental Theorems of Statistical Learning

Theorem 15 (The Fundamental Theorem of Statistical Learning). *Let H be a hypothesis class of functions (may be infinite) from domain X to $\{0, 1\}$ and let the loss function be the 0 – 1 loss. Then, the following are equivalent:*

1. H has the uniform convergence property.
2. Any ERM rule is a successful agnostic PAC learner for H .
3. H is agnostic PAC learnable.

4. H is PAC learnable.
5. Any ERM rule is a successful PAC learner for H .
6. H has a finite VC-dimension.

Theorem 16 (The Fundamental Theorem of Statistical Learning - Quantitative Version). *Let H be a hypothesis class of functions (may be infinite) from a domain X to $\{0, 1\}$ and consider the 0 – 1 loss function. Assume that $VCdim(H) = d < +\infty$. Then there are absolute constants C_1, C_2 such that:*

- H has the uniform convergence property with sample complexity

$$C_1 \frac{d + \log(1/\delta)}{\epsilon^2} \leq m_H^{UC}(\epsilon, \delta) \leq C_2 \frac{d + \log(1/\delta)}{\epsilon^2}$$

- H is agnostic PAC learnable with sample complexity

$$C_1 \frac{d + \log(1/\delta)}{\epsilon^2} \leq m_H(\epsilon, \delta) \leq C_2 \frac{d + \log(1/\delta)}{\epsilon^2}$$

Equivalently: (see the definition of ϵ -representative)

Theorem 17. *Let H be an hypothesis class with VC-dimension $VCdim(H) < +\infty$. Then, with probability $\geq 1 - \delta$ (over the choice of the training set $S \sim D^m$) we have*

$$\forall h \in H, L_D(h) \leq L_S(h) + C \sqrt{\frac{VCdim(H) + \log(1/\delta)}{2m}}$$

where C is a universal constant.

It follows that: if $VCdim(H)$ is finite (and we have enough samples in the training set), then by taking an hypothesis $h \in H$ that minimizes the training error $L_S(h)$, we also get the best hypothesis in the hypothesis class that we're considering in terms of generalization error. This is guaranteed.

Theorem 18. *Let H be a class with $VCdim(H) = +\infty$. Then H is not PAC learnable.*

To summarize: the VC-dimension characterizes PAC learnable hypothesis classes and it also determines the sample complexity.

Chapter 12

Clustering

12.1 Unsupervised Learning

In the supervised learning, the training dataset is $(\mathbf{x}_1, \dots, \mathbf{x}_m)$ with no target values.

We are interested in finding some interesting structure in the data, or, equivalently, to organize it in some meaningful way. The most common approach is *clustering*, and we will use the most commonly used technique called *k-means*.

12.2 Clustering

Informally: the task of finding meaningful groups among data points.

Definition 34. *Clustering is the task of grouping a set of objects such that similar objects end up in the same group and dissimilar objects are separated into different groups.*

The definition above is not rigorous, it may be ambiguous. How can we define similarity between two objects? If the objects are vectors than we can use the distance, if the objects are documents we need another metric.

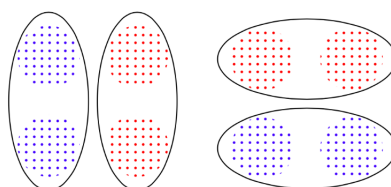


Figure 12.1: Set of objects can be clustered in various different meaningful ways.

12.2.1 Model for clustering

Let's define the model of clustering:

- Input:
 - Set of elements X .
 - Distance function $d : X \times X \rightarrow \mathbb{R}^+$ that is symmetric (i.e. $d(\mathbf{x}, \mathbf{x}') = d(\mathbf{x}', \mathbf{x})$); $d(\mathbf{x}, \mathbf{x}) = 0$ for all $\mathbf{x} \in X$; and d satisfies the triangle inequality $d(\mathbf{x}, \mathbf{x}') \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{x}')$.
- Output:
 - A partition of X into clusters, that is $C = (C_1, \dots, C_k)$ with $\bigcup_{i=1}^k C_i = X$; for all $i \neq j$: $C_i \cap C_j = \emptyset$.

Sometimes the input includes the number k of clusters to produce in output. Sometimes we use a similarity function instead of the distance function. The choice of distance/similarity function depends on the data type; the choice of distances may have an impact on the results.

12.3 Cost Minimization Clustering

A common approach in clustering is to define a cost function over possible partitions of the objects and the goal is to find the partition of minimal cost.

Assumption: data points $\mathbf{x} \in X$ come from a larger space X' , that is $X \subseteq X'$. Define $d(\mathbf{x}, \mathbf{x}')$ as the distance function between $\mathbf{x}, \mathbf{x}' \in X$.

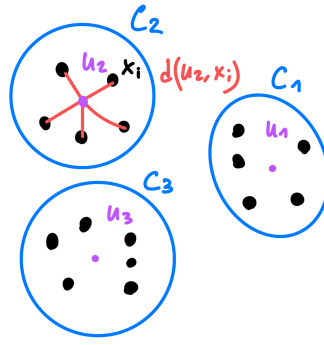
12.3.1 k-Means Clustering

Input: data points $\mathbf{x}_1, \dots, \mathbf{x}_m \in X$ and $k \in \mathbb{N}^+$ (number of cluster).

Goal: find a partition $C = (C_1, \dots, C_k)$ of $\mathbf{x}_1, \dots, \mathbf{x}_m$ and the centers $\mu_1, \dots, \mu_k$ with $\mu_i \in X'$ center C_i with $1 \leq i \leq k$ that minimizes the **k-means objective** (cost):

$$\min \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} d(\mathbf{x}, \mu_i)^2 \quad (12.1)$$

Note that the $\mu_i \in X'$ and $\mathbf{x}_j \in X$ with $X \subseteq X'$, so the centers are not necessarily points taken inside X .



Proposition 14. Given a cluster C_i , the center μ_i that minimizes $\sum_{x \in C_i} d(x, \mu_i)^2$ is

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

i.e. the center of a cluster is the average vector.

12.3.2 k-Medoids objective and k-Median objective

k-medoids objective:

$$\min_{\mu_1, \dots, \mu_k \in X} \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i)^2 \quad (12.2)$$

the only difference between the formula above and the 12.1 is that, in the one above $\mu_i \in X$, i.e., the centres are taken from the data points.

k-median objective:

$$\min_{\mu_1, \dots, \mu_k \in X} \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i) \quad (12.3)$$

an example where such an objective function makes sense is the *facility location problem*.

12.4 Algorithm for k-Means clustering

The brute-force algorithm is to try all possible partitions of the m points into k clusters, evaluate each partition and choose the best one. The problem is that the complexity grows exponentially is proportional to of k^m . Also, finding the optimal solution for k -means clustering is computationally difficult (NP-hard). Therefore we use heuristics methods to find good solution.

Algorithm 19 Lloyd's Algorithm

```

1: Input: data points  $X = \{x_1, x_2, \dots, x_m\}; k \in \mathbb{N}^+$ 
2: Output: clustering  $C = (C_1, C_2, \dots, C_k)$  of  $X$ ; centers  $\mu_1, \mu_2, \dots, \mu_k$ 
3: Randomly choose  $\mu_1^{(0)}, \dots, \mu_k^{(0)}$ 
4: for  $t \leftarrow 0, 1, 2, \dots$  do ▷ until convergence
5:   for  $i = 1, \dots, k$  do
6:      $C_i \leftarrow \{\mathbf{x} \in X : i = \arg \min_j d(\mathbf{x}, \mu_j^{(t)})\}$  ▷ Put  $\mathbf{x}$  in  $C_i$  if  $\mu_i$  is the nearest center
7:   end for
8:   for  $i = 1, \dots, k$  do
9:      $\mu_i^{(t+1)} \leftarrow \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$  ▷ Update the center  $\mu_i$  position (mean value of all  $\mathbf{x} \in C_i$ )
10:  end for
11:  if convergence is reached then
12:    return  $C = (C_1, \dots, C_k)$  and  $\mu_1^{(t+1)}, \mu_2^{(t+1)}, \dots, \mu_k^{(t+1)}$ 
13:  end if
14: end for

```

12.4.1 Lloyd's Algorithm

Commonly used criteria for convergence:

- The k -means objective for the cluster at iteration t is not lower than k -means objective for the cluster at iteration $t - 1$ (i.e. stop when the cost after iteration t is bigger than the cost in iteration $t - 1$).
- The sum of the movement of all centers is lower than ε : $\sum_{i=1}^k d(\mu_i^{(t)}, \mu_i^{(t-1)}) \leq \varepsilon$.
- The maximum movement is lower than ε : $\max_{i \in \{1, \dots, k\}} d(\mu_i^{(t)}, \mu_i^{(t-1)}) \leq \varepsilon$.

If we use the first criteria, then Lloyd's algorithm always terminates.

The time complexity required by the algorithm is $O(tkmd)$ where t is the number of iteration required, k is the number of clusters, m is the number of samples, d is the dimension of vector $\mathbf{x}$.

The number of iteration needed to converge and the quality of the clusters depends on the initialization of the centers. Is there a smart choice of values that can we use to initialize the centers? Yes! See next subsection.

12.4.2 Algorithm k-Means++

Given a point $\mathbf{x} \in X$ and a set F , let $d(\mathbf{x}, F) := \min_{\mathbf{f} \in F} d(\mathbf{x}, \mathbf{f})$. The algorithm:

- **Input:** data points $X = \{x_1, x_2, \dots, x_m\}; k \in \mathbb{N}^+$.
- **Output:** initial set F of centers.

Algorithm 20 Initialization of Centers for K-means++

- 1: $\mu_1 \leftarrow$ random point from X chosen uniformly at random
- 2: $F \leftarrow \{\mu_1\}$
- 3: **for** $i \leftarrow 2$ **to** k **do**
- 4: $\mu_i \leftarrow$ Choose μ_i as a random point from $X \setminus F$, where point $\mathbf{x}$ is chosen with probability

$$\frac{(d(\mathbf{x}, F))^2}{\sum_{\mathbf{x}' \in X \setminus F} (d(\mathbf{x}', F))^2}.$$

The more the distance $d(\mathbf{x}, F)$ the higher the probability.

- 5: $F \leftarrow F \cup \{\mu_i\}$
 - 6: **end for**
 - 7: **return** F
-

A theorem states that the expectation of the cost of the solution provided by the algorithm is not too far from the cost of the optimal solution.

The above algorithm provide already a good solution for k -means but it makes sense to use it for initialize the centers in Lloyd's algorithm in order to improve the solution.

12.5 Choice of number k of clusters

If we can plot the samples, than it is easy to choose the value of k . But if we cannot plot them, how do we choose the value of k ? How can measure if some value of k is better than other values?

One can say: use k -means, start with $k = 1$ and iteratively increase the value. The problem is that the objective function of k -means measures only the distance between the points of a cluster and its center: the lower the better. So, if we increase the number of clusters (i.e. the number of centers) the cost of the objective function will always decrease until we reach the edge case in which we have m cluster (where m is the number of samples in input) and the cost is zero. We need to find a way to measure also the distance between the clusters: the more the better.

Choosing the number of k of clusters (e.g., for k -means) is not easy. A common approach is the following:

1. Run the clustering algorithm for various values of k , obtaining a clustering $C^{(k)} = (C_1^{(k)}, \dots, C_k^{(k)})$ for each value of k considered.
2. Use a score S to evaluate each clustering $C^{(k)}$, getting scores $S(C^{(k)})$ for each value of k .
3. Pick the value of k (and clustering) of maximum score:

$$C = \arg \max_{C^{(k)}} \{S(C^{(k)})\}$$

12.5.1 Silhouette

The silhouette is a very common score based on distance alone.

Let $C = (C_1, \dots, C_k)$ be some partition of X and a point $\mathbf{x} \in X$. Assume that $|C_i| \geq 2 \forall 1 \leq i \leq k$. Define:

$C(\mathbf{x}) :=$ returns the cluster to which $\mathbf{x}$ is assigned to

Define:

$$A(\mathbf{x}) := \frac{\sum_{\mathbf{x}' \neq \mathbf{x}, \mathbf{x}' \in C(\mathbf{x})} d(\mathbf{x}, \mathbf{x}')}{|C(\mathbf{x})| - 1}$$

$$d(\mathbf{x}, C_i) := \frac{\sum_{\mathbf{x}' \in C_i} d(\mathbf{x}, \mathbf{x}')}{|C_i|} \quad \forall C_i \neq C(\mathbf{x})$$

$$B(\mathbf{x}) := \min_{C_i \neq C(\mathbf{x})} d(\mathbf{x}, C_i)$$

where

- $A(\mathbf{x})$ is the mean of the distances between the point $\mathbf{x}$ and all other points that belong to the same cluster of $\mathbf{x}$. Measures the distance between $\mathbf{x}$ and the cluster which it belongs.
- $d(\mathbf{x}, C_i)$ is the mean of the distances between the point $\mathbf{x}$ and all the points in cluster C_i to which $\mathbf{x}$ does not belong. Measures the distance between $\mathbf{x}$ and the cluster C_i .
- $B(\mathbf{x})$ is the distance between $\mathbf{x}$ and the nearest cluster $C_i \neq C(\mathbf{x})$.

We can define the **silhouette** $s(\mathbf{x})$ of $\mathbf{x}$:

$$s(\mathbf{x}) = \frac{B(\mathbf{x}) - A(\mathbf{x})}{\max\{A(\mathbf{x}), B(\mathbf{x})\}} \quad (12.4)$$

The value of $s(\mathbf{x})$ measures if $\mathbf{x}$ is closer to points in its "nearest cluster" than to the cluster it is assigned to. As we can see $s(\mathbf{x}) \in [-1, 1]$: if $s(\mathbf{x}) = -1$ then $\mathbf{x}$ is very far from the cluster it is assigned to; if $s(\mathbf{x}) = 1$ then $\mathbf{x}$ is very close to the cluster it is assigned to and very far from the nearest one.

We can define the silhouette of clustering $C = (C_1, \dots, C_k)$ (the score of C) as follows:

$$S(C) = \frac{\sum_{\mathbf{x} \in X} s(\mathbf{x})}{|X|}$$

the mean of silhouette computed for each $\mathbf{x} \in X$. The higher $S(C)$, the better the clustering quality.